A BDF-Compatible Finite Element Solver in Julia:
Architecture, Solution Sequences, and Differentiable Extensions

JFEM

Universidad de Sevilla

Updated June 12, 2026

Abstract

This paper documents the Julia finite element solver contained in the present repository and explains its organization as a compact, BDF-compatible analysis code for structural mechanics. The implementation follows a three-stage workflow: (i) translation of a Bulk Data File (BDF) into a JSON-serializable internal model, (ii) execution of a solution sequence on sparse system matrices, and (iii) export of results to VTK, JSON, HDF5, and a custom JFEM binary format. The checked-in code explicitly supports linear static analysis (SOL 101), normal modes (SOL 103, with legacy SOL 63 normalized into the same modal path), linear buckling (SOL 105), and an experimental geometrically nonlinear static branch exposed through SOL 106. The parser resolves recursive INCLUDE cards, accepts fixed-field, free-field, tab-delimited, and large-field decks, and assembles a model that includes shell, bar, beam, rod, spring, rigid, mass, and solid entities together with composite laminates derived from PCOMP cards.

Beyond forward analysis, the repository implements an adjoint sensitivity framework for static responses and buckling eigenvalues, plus restartable sizing workflows. A key architectural feature is the derivative back-end registry in the solver: some derivatives are analytical, others are obtained by localized finite differences or full-model reassembly, and the design now includes package-backed automatic-differentiation paths through ForwardDiff for line-element area sizing and shell-material differentiation. In the current implementation the shell-thickness and shell Poisson-ratio routes use ForwardDiff on both `CTRIA3` and the default `CQUAD4` operators employed by the static adjoint path, while PCOMP ply-thickness and ply-angle sensitivities now default to exact laminate-level derivatives of the stored CLT matrices, with the older CLT finite-difference path retained as a fallback. On the buckling side, the shell $\partial \mathbf{K}_G/\partial h$ contraction is evaluated exactly on the standard shell path and through branch-aware local finite differences on the newer specialized isotropic quadrilateral subsets, while the shell $\partial \mathbf{K}_G/\partial \nu$ contraction remains exact on the standard and flat specialized shell subsets and is now also synchronized through branch-aware fixed-$\mathbf{u}$ local contractions on the verified explicit covariant-operator isotropic quadrilateral subset. The adjoint now contains branch-aware quadrilateral-shell linearizations for flat orthotropic PCOMP buckling branches, the flat isotropic quadrilateral subset used by the solver's exact-membrane SOL 105 path, the flat-curved isotropic exact-membrane subset exercised by faceted cylinder panels and small barrel and tapered-cylinder decks, an explicit covariant isotropic `CQUAD4` subset that now includes the curved geomnormal-frame covariant-operator branch, and a verified compatible-membrane auto-covariant isotropic `CQUAD4` membrane-recovery subset. Recent refactoring also separates the static and buckling-eigenvalue stiffness operators in the SOL 105 adjoint whenever full-model reassembly is needed, so the implementation contracts $\boldsymbol{\phi}^{\mathsf{T}} \partial \mathbf{K}_{\mathrm{eig}}/\partial x\, \boldsymbol{\phi}$ and $\boldsymbol{\psi}^{\mathsf{T}} \partial \mathbf{K}_{\mathrm{static}}/\partial x\, \mathbf{u}$ on their matching operators instead of reusing a single matrix for both terms. The current solver now also caches those reassembled stiffness- and geometric-stiffness-derivative operators and the static-state pseudo-load groups across modes for a given design variable, which reduces the cost of the

remaining full-model buckling paths without changing the returned gradients. For the remaining hardest exact-membrane curved-shell groups, the buckling sensitivity driver still retains a narrow mode-tracked full-model finite-difference fallback for the complete eigenvalue derivative so that the returned group sensitivities stay synchronized with the forward branch even when a fully local adjoint decomposition would be less reliable. The buckling RHS linearization on the specialized isotropic quadrilateral branches now freezes the active prestress-field selector, including residual, Gauss-point, and blended sigma-field choices, during basis probing. The optimization driver now covers both shell thickness and bar area, and the validation harness includes composite mode-tracking smoke tests together with focused finite-difference checks on the specialized buckling RHS, the flat-isotropic $\partial \mathbf{K}_G / \partial \nu$ contraction, the flat-curved isotropic exact-membrane cylinder-panel subset, the explicit covariant isotropic subset, the compatible-membrane auto-covariant membrane-recovery branch, split-PID shell-thickness checks on the flat-curved, explicit-covariant, and auto-covariant isotropic quadrilateral subsets, a split-MID Poisson-ratio check on the explicit covariant-operator subset, dedicated curved-shell buckling-adjoint finite-difference checks on the exact-membrane barrel and tapered-cylinder families, and new MAC-based curved-shell regressions on barrel, dome, and tapered-cylinder families. The goal of this manuscript is therefore not to claim a complete clone of commercial Reference, but to explain with precision what the current code does, how it does it, and where its advanced capabilities are most useful.

Keywords: finite elements; Julia; BDF; linear static analysis; geometric nonlinearity; normal modes; buckling; adjoint sensitivities; composite laminates

1 Introduction

Open structural solvers that speak a BDF-based input language occupy a useful middle ground between teaching codes and full industrial systems. They allow existing benchmark decks to be reused, make cross-validation against reference solvers practical, and expose the numerical machinery in a form that can be inspected and extended. The repository studied here follows exactly that philosophy. Its scope is deliberately narrower than a full general-purpose Reference implementation, yet within that scope it is unusually transparent: the parse model is JSON-serializable, the analysis stages are cleanly separated, the supported solution sequences are explicit, and post-processing is first-class rather than an afterthought.

The code base is also notable for its treatment of sensitivities. Instead of bolting gradient computation onto a finished solver, the repository organizes design-variable derivatives through a shared pseudo-load interface and couples that interface to static and buckling adjoint solvers. This makes the solver useful not only for forward simulation but also for parameter studies, laminate tailoring, geometry perturbation, and simple optimization experiments.

This manuscript was refreshed against the current OpenJFEM workspace on 2026-06-12, including the public modules under `src/`, private code tools under the development harness, and private validation summaries under the validation archive. The emphasis is therefore on implementation truth: every capability described below is grounded in the repository as it exists in this workspace.

2 Software Architecture

At the top level the solver is organized around a three-stage workflow exposed by `JFEM/src/main.jl` and `JFEM/src/JFEMSolver.jl`. A BDF deck is first converted into a model dictionary, that

dictionary is then solved without file I/O, and the results are finally exported to machine- and visualization-friendly formats. The internal model itself is intentionally plain: it is a nested Julia `Dict` that can also be serialized to JSON for cross-language use. The checked-in `main.jl` file now also keeps its command-line execution path behind a direct-execution guard, so testing or helper scripts can `include` the same source tree without accidentally launching the default example deck.

Figure 1 summarizes the dataflow. The parsing layer resolves INCLUDE files, splits executive, case-control, and bulk-data sections, normalizes card formats, and builds a model dictionary containing geometry, properties, materials, constraints, loads, and case-control metadata. The solver layer assembles sparse operators and dispatches to SOL 101, 103, 105, or the current experimental SOL 106 branch, with legacy SOL 63 decks entering the same modal branch as SOL 103. The export layer writes the final state to VTK, aggregated JSON, aggregated or solution-specific HDF5 payloads, modal or buckling eigenvalue JSON, card inventories, and the JFEM binary format. If an `adjoint_config.json` file is found next to the input deck, the code also launches a sensitivity pass after the forward solve; optimization runs can additionally emit restart checkpoints containing sizing histories and solver diagnostics.

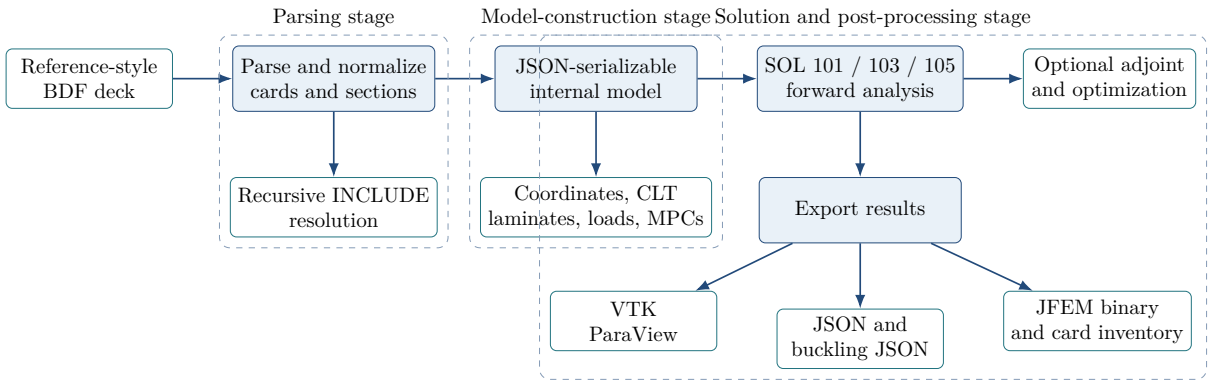


Figure 1: Repository-level software architecture. The checked-in implementation is organized as parse, solve, and export stages, with optional adjoint and optimization hooks attached after the forward solution.

Table 1 maps the main source files to their roles. This mapping is useful because the repository is already modular enough that most extension work can be localized to one or two files.

3 Input Language and Model Construction

3.1 Deck structure

The parser follows the classical Reference deck partition into executive control, case control, and bulk data. The code recognizes `SOL` in the executive section, honors `CEND`, and detects `BEGIN BULK` and `ENDDATA`. When a deck omits explicit subcases but provides global `LOAD` or `SPC` definitions, the parser automatically synthesizes a default `SUBCASE 1`. This behavior is implemented in `src/parsing/card_processor.jl` and makes smaller benchmark decks easier to run.

The repository is liberal in the card syntax it accepts. The low-level utilities in `src/parsing/utilities.jl` handle

- fixed-field cards,

Table 1: Principal source modules and their roles.

File or module	Primary responsibility
`src/main.jl`	Entry point, package loading, parse–solve–export orchestration, optional adjoint trigger when `adjoint_config.json` exists.
`src/JFEMSolver.jl`	Public API for `bdf_to_model`, `solve_model`, export helpers, and solution-sequence dispatch.
`src/parsing/*.jl`	BDF utilities, card processing, include resolution, and extraction of geometry, properties, materials, loads, and constraints.
`src/ModelBuilder.jl`	Construction of the solver-ready model dictionary, coordinate resolution, geometry transformation, and PCOMP laminate reduction to CLT matrices.
`src/FEMKernels.jl`	Element kernels for shells, beams, rods, springs, and solids, plus geometric and mass matrices.
`src/solver/assembly.jl`	Sparse stiffness and geometric-stiffness assembly, shell enrichment switches, higher-order shell subdivision, DMIG injection, and solid support.
`src/solver/solve_case.jl`	Static solve orchestration, experimental nonlinear-static load stepping, modal and buckling eigensolvers, mass-matrix assembly, and structured solver diagnostics for direct or Krylov branches.
`src/solver/boundary_conditions.jl`	SPC handling, AUTOSPC logic, mechanism detection, factorization fallback, and DOF recovery.
`src/solver/constraints.jl`	RBE2, RBE1, RBE3, RSPLINE, and MPC elimination through stiffness redistribution.
`src/solver/adjoint.jl`, `src/solver/adjoint_buckling.jl`, `src/solver/dKdx.jl`	Static and buckling adjoint sensitivities plus derivative back-end dispatch.
`src/solver/optimize_thickness.jl`	Thickness and generalized sizing optimization using optimality criteria for compliance and projected gradients for buckling, with checkpoint/restart support and structured iteration histories.
`src/Export.jl`	VTK, JSON, HDF5, JFEM binary, buckling export, and card-inventory output.

- free-field cards separated by commas,
- tab-delimited variants that appear in legacy validation decks,
- large-field cards denoted by an asterisk,
- recursive INCLUDE resolution with relative or absolute paths.

INCLUDE nesting is handled recursively with a depth guard, so split model databases can be analyzed without pre-flattening the deck.

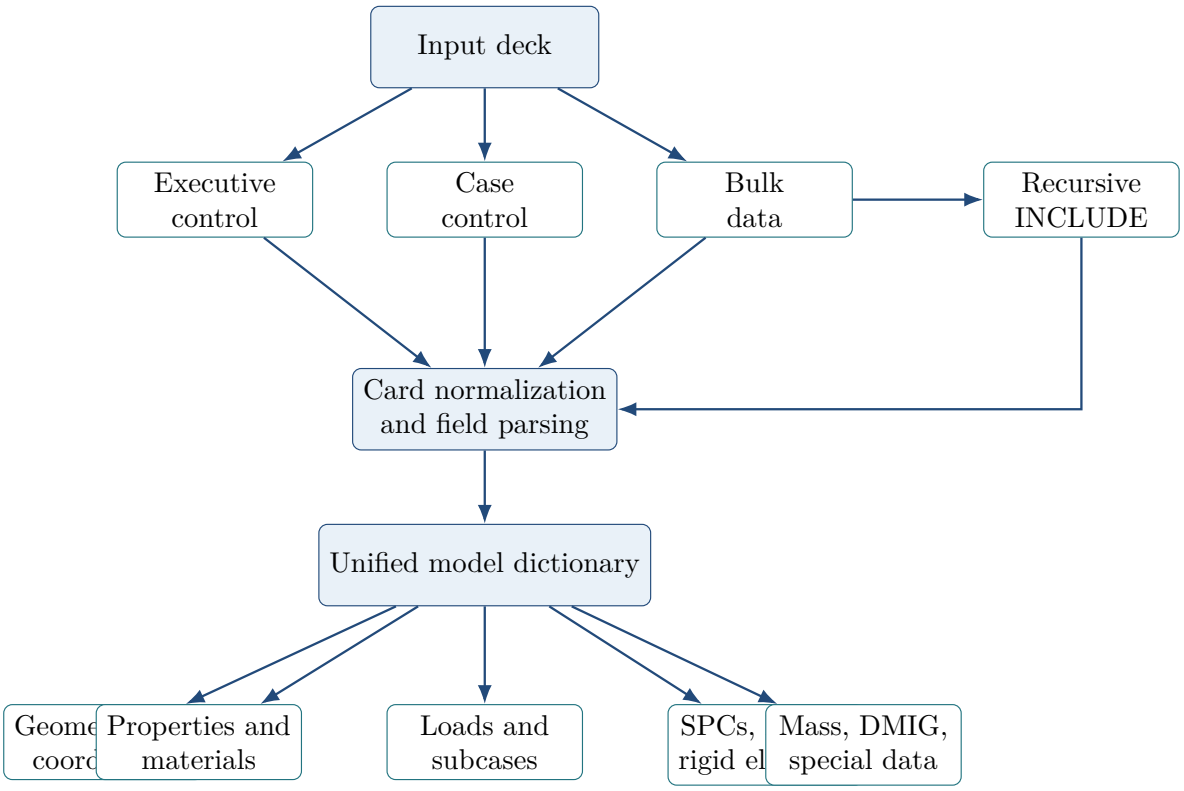


Figure 2: Conceptual view of the accepted input structure. The parser resolves INCLUDE chains, extracts case-control directives, and then builds a unified internal model from the bulk-data cards.

3.2 Supported cards and categories

The code explicitly inventories processed card types during import and writes a `*.CARDS.JSON` file so that unsupported content is visible rather than silently ignored. Table 2 groups the major card families that are consumed by the checked-in implementation.

Two implementation details are especially important.

First, composite laminates described by `PCOMP` are converted in `src/ModelBuilder.jl` into classical laminate theory (CLT) membrane, bending, coupling, and shear matrices. The code stores those matrices in a shell-property dictionary together with per-ply transformed constitutive data and the originating ply material identifiers, which are later reused in stress recovery and in exact derivative calculations with respect to ply thickness or angle.

Table 2: Major BDF entities recognized by the repository.

Category	Examples
Geometry and coordinates	GRID, GRDSET, CORD1R, CORD2R, CORD2C, CORD2S. Nested coordinate systems are resolved before analysis, and GRDSET defaults are folded into the derived grid records.
Shell and solid elements	CTRIA3, CTRIA6, CQUAD4, CQUAD8, CSHEAR, CTETRA, CHEXA, CPENTA. Higher-order shells are subdivided into lower-order sub-elements during assembly.
Line, spring, and bushing elements	CBAR, CBEAM, CROD, CONROD, CELAS1, CELAS2, CBUSH.
Properties and materials	PSHELL, PSHEAR, PCOMP, PBARL, PBAR, PBEAM, PBEAML, PROD, PELAS, PBUSH, PSOLID, MAT1, MAT2, MAT8, MATT1, TABLEM1.
Constraints and interpolation	SPC1, SPC, SPCADD, MPC, MPCADD, RBE1, RBE2, RBE3, RBAR, RSPLINE.
Loads and analysis control	FORCE, MOMENT, PLOAD, PLOAD1, PLOAD2, PLOAD4, GRAV, RFORCE, LOAD, TEMP, TEMPD, EIGRL, PARAM, DMIG.
Mass and inertia	CONM1, CONM2, CMASS1, CMASS2, PMASS.

Second, several cards are normalized into a unified internal representation. For example, PLOAD2 is folded into the pressure-load list, RBAR is converted to an RBE2-equivalent constraint, GRDSET defaults are folded into the derived GRID records, and MAT8 laminates receive equivalent scalar properties for downstream procedures that need isotropic-like bookkeeping.

3.3 Representative input example

Listing 1 shows a shortened representative shell deck in the same style as the checked-in validation examples. The deck combines executive control, a subcase, material and property definitions, shell elements, SPCs, and nodal forces.

Listing 1: Representative BDF structure used by JFEM examples and validation decks.

```
SOL 101
CEND
TITLE = CANTILEVER 4x8
SUBCASE 1
  LOAD = 1
  SPC = 1
  DISPLACEMENT = ALL
BEGIN BULK
MAT1    1       2.1+11          0.3     7850.0
PSHELL  1       1       0.005    1               1
GRID    1               0.0     0.0     0.0
GRID    2               0.25    0.0     0.0
...
CQUAD4  1       1       1       2       7       6
...
SPC1    1       123456  1       THRU    5
FORCE   1       41      0       1.0     0.0     0.0     -62.5
...
ENDDATA
```

4 Finite Element Formulation and Solution Sequences

4.1 Discretization choices

The solver uses six structural degrees of freedom per shell or line node and three translational degrees of freedom per solid node. Several distinct element families coexist in the same global assembly:

- shell elements based on triangular and quadrilateral Reissner–Mindlin kinematics,
- bar and beam elements with consistent local-frame transformations and optional offsets,
- rod and spring elements,
- rigid or interpolation constraints handled by DOF elimination,
- three-dimensional solid elements `CTETRA4`, `CHEXA8`, and `CPENTA6`.

The shell implementation is one of the most developed parts of the code. The kernels in `src/FEMKernels.jl` and `src/solver/assembly.jl` contain selective reduced integration, incompatible-mode toggles, curved-shell normal handling, drill-stiffness controls, and multiple heuristics specific to buckling accuracy. In practical terms, the repository behaves like a research-grade shell code with a Reference-style front-end rather than a minimal educational plate element.

Higher-order shell cards are currently supported through subdivision rather than dedicated high-order kernels: each `CQUAD8` is expanded into four `CQUAD4`-type sub-elements and each `CTRIA6` into four `CTRIA3`-type sub-elements. This preserves input compatibility and mesh coverage at modest implementation cost.

4.2 SOL 101: linear static analysis

The static sequence solves the symmetric equilibrium problem

$$\mathbf{K}\mathbf{u} = \mathbf{F}, \tag{1}$$

where $\mathbf{K}$ is the assembled global stiffness matrix, $\mathbf{u}$ is the analysis-frame displacement vector, and $\mathbf{F}$ is the load vector produced by the load-resolution subsystem.

Load assembly in `src/solver/loads.jl` includes nodal forces and moments, pressure loads on shells and solid faces, bar distributed loads, gravity, centrifugal loading through `RFORCE`, recursive `LOAD` combinations, and thermal loading from `TEMP`/`TEMPD`. The `TEMP` extractor now reads repeated (G, T) pairs across continuation fields rather than stopping after the first three pairs. The same temperature sets are also routed through `TEMP(LOAD)` subcase selectors; on the present isotropic `MAT1`-backed line, shell, and solid paths, `MATT1`/`TABLEM1` can therefore reassemble the active subcase stiffness and prestress operators from the referenced temperature field. Boundary conditions combine permanent node constraints inherited from `GRID`/`GRDSET` `PS` fields, explicit SPC sets, optional SPCADD expansion, elimination of dependent DOFs from rigid or MPC relations, and AUTOSPC detection for disconnected or numerically singular DOFs.

The static linear solve is handled in `src/solver/boundary_conditions.jl`. The preferred path is Cholesky factorization of the reduced system. If factorization reveals mechanisms, the code

performs a second-stage factorization-based AUTOSPC pass; if that still fails, the solver falls back to LU. For very large systems, the implementation can switch to conjugate-gradient or MINRES iterations with algebraic multigrid preconditioning.

Post-processing recovers displacements in global coordinates, SPC reactions, shell forces, shell stresses, bar and rod forces, spring forces, and solid stresses. An equilibrium check comparing applied loads and accumulated reactions is printed as part of the static solve.

4.3 SOL 103: normal modes

The modal sequence assembles a consistent mass matrix and solves

$$\mathbf{K}\boldsymbol{\phi} = \omega^2 \mathbf{M}\boldsymbol{\phi}, \tag{2}$$

with ω the circular frequency and $\boldsymbol{\phi}$ the mode shape. The mass matrix includes distributed shell, line, and solid mass, non-structural mass where the present property route provides it, concentrated masses, inertia offsets for `CONM2`, and the `WTMASS` conversion factor when present.

Case-control data provide the modal extraction parameters through `METHOD` and `EIGRL`. The solver honors the number of requested modes (`ND`) and frequency filtering bounds (`V1`, `V2`). For systems with roughly up to 4000 free DOFs it uses a dense generalized eigensolver; for larger systems it switches to a KrylovKit shift-invert strategy. If the modal case activates `TEMP(LOAD)` together with temperature-dependent `MAT1` data, the dispatcher now reassembles the active isotropic `MAT1` stiffness and the current shell, line, and solid mass contributions under that temperature field before eigenvalue extraction. Returned mode shapes are mass-normalized and exported together with total mass and modal effective mass in the global x, y, and z directions. At the API level, the dispatcher canonicalizes legacy SOL 63 requests into the same modal family, which simplifies reuse of older benchmark decks and harmonizes downstream validation.

4.4 SOL 105: linear buckling

The buckling sequence is implemented as a classical prestressed eigenvalue problem. A reference static state is computed first, from which the code forms the geometric stiffness matrix associated with the prestress field. The subsequent eigenproblem is written as

$$(\mathbf{K} + \lambda \mathbf{K}_G)\,\boldsymbol{\phi} = \mathbf{0}, \tag{3}$$

or, equivalently,

$$\mathbf{K}\boldsymbol{\phi} = \lambda\,(-\mathbf{K}_G)\,\boldsymbol{\phi}, \tag{4}$$

where λ is the buckling load factor and $\boldsymbol{\phi}$ is the buckling mode.

This organization matches the implementation in `src/solver/solve_case.jl`: the solver first executes a linear static preload solve, then assembles $\mathbf{K}_G$ from the element stress resultants, and finally extracts eigenpairs according to the request in `EIGRL`. If the preload subcase activates `TEMP(LOAD)` together with temperature-dependent `MAT1` data, the static reference solve, the linear buckling operator, and the geometric-stiffness assembly are all re-evaluated under that same subcase temperature field for the current isotropic line, shell, and solid paths. For modest problem sizes a dense symmetric-definite eigensolver is preferred; if its assumptions are violated, the code falls back to a more general dense solver. For larger systems the implementation switches to Krylov-based inverse or shifted iteration through KrylovKit.

In shell-dominated models the buckling path contains several practical controls that matter numerically: drilling stabilization, optional membrane–bending couplings, selective integration options, and geometric-stiffness heuristics tuned for plate and shell behavior. This is one reason the repository performs well on many classical panel and shell buckling benchmarks despite its compact size.

4.5 SOL 106: experimental geometric nonlinear static analysis

The current source tree now exposes an experimental nonlinear-static branch through SOL 106. The implementation is intentionally conservative in how it is described: it is not a full industrial large-displacement formulation, but it now separates two distinct nonlinear routes rather than treating every case with the same approximate update. The default route remains a load-stepped geometric-stiffness iteration that reuses the existing linear static and prestress infrastructure. At load step n and iteration k, that legacy branch assembles a tangent-like operator of the form

$$\mathbf{K}_{\mathrm{eff}}^{(k)} = \mathbf{K} + \mathbf{K}_G\Big(\mathbf{u}^{(k)}\Big), \tag{5}$$

and then forms a correction equation on the current residual rather than recomputing a full total-load state at every inner iteration. In that legacy branch the correction tangent is the linear-plus-geometric operator $\mathbf{K} + \mathbf{K}_G$, while the residual model is configurable. The current cantilever-oriented regressions exercise a secant-style option of the form $\mathbf{K}\mathbf{u} + \frac{1}{2}\mathbf{K}_G(\mathbf{u})\mathbf{u} - \lambda_n\mathbf{F}$, and the same infrastructure also supports the tangent-style residual $(\mathbf{K} + \mathbf{K}_G)\mathbf{u} - \lambda_n\mathbf{F}$.

In addition, the source tree now exposes an explicit formal branch through the nonlinear-method parameter `PARAM_NLMETHOD`, with the current formal option selected as `formal_shell_von_karman`. The same `TEMP(LOAD)` routing used by the linear solver is now threaded through both nonlinear routes for the current isotropic `MAT1`-supported line, shell, and solid stiffness paths. On the presently supported formal flat-shell subset, the shell membrane constitutive matrix is likewise rebuilt from the active temperature-adjusted `MAT1` state when such data are available. For the currently supported flat coplanar `CTRIA3`/`CQUAD4` `PSHELL` subset, this branch assembles a shell-level von Kármán membrane contribution $\mathbf{f}_{\mathrm{vk}}(\mathbf{u})$ and its consistent tangent,

$$\mathbf{r}^{(k)} = \mathbf{K}\mathbf{u}^{(k)} + \mathbf{f}_{\mathrm{vk}}\Big(\mathbf{u}^{(k)}\Big) - \lambda_n\mathbf{F}, \qquad \mathbf{K}_{\mathrm{eff}}^{(k)} = \mathbf{K} + \left.\frac{\partial \mathbf{f}_{\mathrm{vk}}}{\partial \mathbf{u}}\right|_{\mathbf{u}^{(k)}}. \tag{6}$$

This is a materially different route from the earlier geometric-stiffness residuals because the nonlinear force vector and tangent come from an explicit shell internal-force construction rather than from a reused prestress operator. The formal route is still deliberately scoped: it is opt-in, it is currently limited to the flat-shell subset just described, and the solver keeps `:auto` on the legacy branch until the same level of robustness is demonstrated on broader bending-dominated and curved-shell cases.

Both branches use the same nonlinear control scaffolding. The tentative correction is filtered through a backtracking line search, and a load step is declared converged only after the accepted iterate falls below the residual tolerance and also satisfies either a relative displacement-change tolerance or a normalized incremental-work tolerance. On the formal branch the driver now evaluates the allowed backtracking candidates, keeps the larger Armijo-admissible step when every admissible trial still worsens the residual, but prefers a later admissible candidate once the residual stops worsening and a smaller accepted trial yields a cleaner residual reduction. If the largest formal trial already delivers a residual-based acceptance, whether through direct residual reduction or because

the residual tolerance is already met, the remaining smaller backtracking candidates are skipped because they no longer affect the convergence decision enough to justify extra shell-state evaluations. The same short-circuit now applies once the formal residual is already inside a near-converged band: after that point the first residual-reducing candidate is accepted immediately instead of continuing to scan smaller Armijo-admissible trials that no longer change the load-step decision materially. The implementation also avoids redundant inner-loop work by reusing the already computed free/fixed DOF partition from the current nonlinear state inside the correction solve and by constructing the final static-style outputs directly from the accepted nonlinear state, rather than by re-solving a separate tangent system for postprocessing. The nonlinear driver treats the user-specified number of load steps as an adaptive nominal increment schedule rather than a rigid partition: if a step does not satisfy the convergence criteria within the iteration budget, the increment is reduced by a configurable cutback factor and retried from the last converged state, while accepted steps are now allowed to enlarge the next increment only when the recorded accepted iterations stayed on full-size updates, reduced the residual monotonically, and avoided fallback-style trial selection. Recent robustness work also replaces the earlier geometric-mean cutback recovery shortcut with a contraction-informed bracket controller: after an accepted cutback step, the next increment is still limited by the bracket defined by the accepted cutback size and the larger failed request, but the allowed recovery inside that bracket is scaled by the actual residual contraction achieved by the accepted retry rather than by a fixed interpolation rule alone. The convergence test itself now stays on the same residual-plus-update-or-incremental-work basis even when accepted steps become very small, rather than switching to a separate machine-scale shortcut. The nonlinear diagnostics now also expose how often accepted steps fall below a tiny-increment reporting threshold, how many of those small steps were cutback-driven, how many of them exited back to the nominal schedule on the following accepted step, the length of the longest tiny-step plateau, the largest recovery ratios applied after accepted cutback steps, the largest residual-contraction and proposed-recovery ratios observed inside that controller, which larger failed increment served as the recovery reference for each accepted cutback step, how many cutback retries had already satisfied the residual tolerance before being reduced again, and explicit plateau summaries that record where each tiny-step streak started, how many cutback-driven steps it contained, how many nonlinear iterations and line-search trials/backtracks it consumed locally, how many residual-satisfied retries accumulated inside that streak, and how it finally exited. The resulting diagnostics now record which nonlinear method was active, which residual and correction-tangent model were used, why each accepted line-search trial was taken (residual tolerance, residual reduction, Armijo decrease on the formal branch, or best-available fallback), which selection policy chose the final accepted trial on each iteration, whether the accepted iterate actually reduced the residual, internal-force, and, where applicable, potential-energy metrics, which correction solver was used, whether the current-state partition was reused, how many line-search trials and backtracks were consumed, what termination reason closed the step, whether a cutback was required, whether the next step was allowed to grow, the final relative change and normalized incremental-work measures, growth-eligibility flags derived from the accepted step history, and, on the formal branch, which backtracking trial had the smallest raw merit among all recorded candidates versus which admissible trial actually governed the accepted update. The same formal diagnostics now also summarize, at load-step and subcase level, how often the accepted trial differed from the raw minimum-merit candidate or from the minimum admissible candidate and how large those trial-index gaps became. Those paired merit-reference diagnostics make it possible to compare future generic globalization rules against the currently validated selector without changing the solver path blindly, while still reporting the shell count and operator size of the final formal tangent contribution.

Table 3 summarizes the nonlinear control parameters that are explicitly read by the current dispatcher in `src/JFEMSolver.jl`. This is useful in practice because the SOL 106 behavior is configured entirely through model-level `PARAM_*` entries rather than through a separate nonlinear input block.

Table 3: Model parameters currently read by the SOL 106 nonlinear dispatcher. Defaults are the values used when the corresponding `PARAM_*` entry is absent.

Parameter	Default	Role in the current implementation
`PARAM_NLMETHOD`	`auto`	Selects the nonlinear branch. The checked-in alternatives are the legacy geometric route and the explicit flat-shell `formal_shell_von_karman` branch.
`PARAM_NLRESMODEL`	`tangent_operator`	Chooses the residual model on the legacy geometric branch: tangent-style $(\mathbf{K}+\mathbf{K}_G)\mathbf{u}-\lambda\mathbf{F}$ or the secant-style variant used by the cantilever regressions.
`PARAM_NLLOADSTEPS`	4	Sets the nominal number of load increments before any adaptive cutback or growth modifies the schedule.
`PARAM_NLMAXITER`	8	Maximum nonlinear correction iterations allowed per attempted load step.
`PARAM_NLTOL`	10^{-6}	Relative displacement-change tolerance for step convergence.
`PARAM_NLRESTOL`	`PARAM_NLTOL`	Relative residual tolerance. If omitted, the residual criterion defaults to the same numerical value as `PARAM_NLTOL`.
`PARAM_NLRELAX`	1.0	Initial scale factor applied to the correction before any line-search backtracking; the implementation clamps it to the interval $[10^{-3}, 1]$.
`PARAM_NLLSMAX`	6	Maximum number of line-search backtracks, so each iteration may evaluate up to `PARAM_NLLSMAX`+1 trial steps.
`PARAM_NLLSREDUCE`	0.5	Multiplicative reduction factor applied between successive line-search trials.
`PARAM_NLCUTMAX`	6	Maximum number of load-step cutback retries after a failed attempt.
`PARAM_NLCUTREDUCE`	0.5	Multiplicative reduction factor used when a failed load step is retried with a smaller increment.
`PARAM_NLSTEPGROW`	1.25	Upper growth factor used to enlarge the next nominal increment after a smoothly converged step.

The implementation lives primarily in `src/solver/solve_case.jl`, with the public dispatch in `src/JFEMSolver.jl`. It records step-by-step nonlinear diagnostics, including per-step convergence flags, iteration histories, relative update norms, normalized incremental-work measures, residual norms before and after the accepted step, internal-force and geometric-force norms, the chosen nonlinear method and residual model, explicit line-search acceptance reasons, selected-trial indices, selection-policy labels, and reduction flags, the correction-solver backend and residual for each iteration, whether the current-state partition was reused, accepted line-search scales, cutback counts, per-step attempt histories, accepted versus requested load-step counts, next-increment reasons, explicit termination reasons, counts of nonlinear state evaluations and accepted-state reuses, cumulative line-search trial and backtrack counts, counts of adaptive step-growth events, accepted-versus-

merit trial-divergence counts and maximum gaps on the formal branch, and the final geometric-stiffness or formal-tangent summary. The purpose of this path is twofold. First, it provides a practical nonlinear scaffold for regression testing and workflow prototyping. Second, it creates explicit extension points for broader total-Lagrangian, Newton-consistent, and formal shell nonlinear developments without disturbing the established SOL 101/103/105 branches.

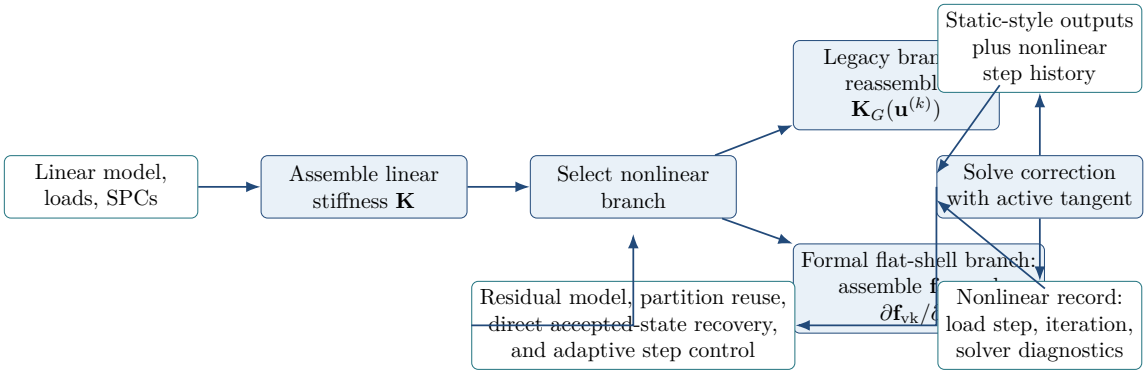


Figure 3: Current experimental SOL 106 workflow. A baseline linear stiffness matrix is assembled first; if a nonlinear subcase activates `TEMP(LOAD)` together with temperature-dependent `MAT1` data, the corresponding isotropic line, shell, and solid operators are reassembled under that subcase temperature field before the nonlinear solve begins. The nonlinear branch then either reassembles a geometric stiffness from the current displacement iterate or, on the currently supported flat-shell subset, assembles a formal von Kármán shell contribution and its consistent tangent. The correction equation is solved with the active tangent using the current-state boundary-condition partition. The tentative update is accepted through a residual-controlled line search; if the step does not converge within the iteration budget, the increment is cut back and retried from the last converged state, while smoothly converged steps can enlarge the next nominal increment.

4.6 Sequence summary

Table 4 condenses the currently exposed solution sequences.

5 Output and Interoperability

The separation between `solve_model` and export routines is a strong design choice in this code base. Because the forward solve returns a structured result dictionary, the same analysis state can be routed to several downstream formats without re-solving. The exporter in `src/Export.jl` currently writes

- `*.JU.JSON` files for aggregate static results,
- `*.JU.H5` files for the HDF5 counterpart of those aggregated static exports,
- `*.NONLINEAR.JSON` files for dedicated SOL 106 nonlinear histories and summaries,
- `*.NONLINEAR.H5` files for dedicated SOL 106 nonlinear histories and summaries in HDF5 form,

Table 4: Solution sequences explicitly dispatched by the checked-in solver.

SOL	Primary equation	Typical control cards	Principal exported quantities
101	$\mathbf{K}\mathbf{u} = \mathbf{F}$	`LOAD`, `SPC`/`SPCADD`, optional `PARAM` entries	Displacements, reactions, shell forces and stresses, line-element forces, solid stresses, VTK fields, aggregate JSON.
103 / 63	$\mathbf{K}\boldsymbol{\phi} = \omega^2\mathbf{M}\boldsymbol{\phi}$	`METHOD`, `EIGRL`, mass cards, optional `WTMASS`	Frequencies, mode shapes, total mass, modal effective masses, VTK mode output, and eigensolver diagnostics.
105	$\mathbf{K}\boldsymbol{\phi} = \lambda(-\mathbf{K}_G)\boldsymbol{\phi}$ after prestress solve	`LOAD`, `SPC`, `METHOD`, `EIGRL`	Buckling load factors, mode shapes, preload state, buckling JSON, VTK mode output, optional adjoint eigenvalue sensitivities.
106	$\mathbf{K}_{\text{eff}}^{(k)}\Delta\mathbf{u}^{(k)} = -\mathbf{r}^{(k)}$ with either $\mathbf{K}_{\text{eff}}^{(k)} = \mathbf{K} + \mathbf{K}_G(\mathbf{u}^{(k)})$ on the legacy branch or $\mathbf{K}_{\text{eff}}^{(k)} = \mathbf{K} + \partial\mathbf{f}_{\text{vk}}/\partial\mathbf{u}$ on the current flat-shell formal branch, together with residual-controlled correction updates, partition reuse, and adaptive cutback/growth	`LOAD`, `SPC`, optional nonlinear control parameters stored as model `PARAM_*` entries, including `PARAM_NLMETHOD` for the explicit formal flat-shell branch	Static-like displacements, reactions, stresses, VTK export, aggregated `.JU.JSON` export with embedded nonlinear histories, a dedicated `.NONLINEAR.JSON` summary, and structured nonlinear step diagnostics including method, residual-model, correction-solver, residual, force-norm, line-search, cutback, growth, termination, and formal-operator records.

- `*.MODES.H5` and `*.BUCKLING.H5` files for modal and buckling eigenpairs plus raw mode-shape arrays,
- `*.BUCKLING.JSON` files for modal or buckling eigenpairs, tagged by analysis type,
- `*.OPTIMIZATION.H5` files for SOL 200-lite optimization payloads,
- `*.ADJOINT.JSON` files for sensitivity output when adjoint calculations are requested,
- `*.vtu` files for ParaView-compatible visualization,
- `*.jfem` files in the repository's custom binary format,
- `*.CARDS.JSON` inventories that record which card types were parsed.

The VTK path is especially useful for inspection of shell forces, stress patterns, and eigenmodes in external visualization tools. The JFEM binary writer, by contrast, is aimed at compact interchange with other programs that may want direct access to mesh, property, and result arrays without reparsing a BDF deck.

The current implementation also enriches the machine-readable exports with structured solver diagnostics. For SOL 101 these diagnostics include boundary-condition partitioning, AUTOSPC actions, factorization fallback behavior, residual norms, and equilibrium summaries. The boundary-condition block now also records how many permanent DOFs came from `GRID/GRDSET PS` fields before any explicit SPC sets were applied. For SOL 103 and SOL 105 the diagnostics include the eigensolver backend, requested and returned mode counts, range-filter information, and recorded fallback attempts. Modal output has also been upgraded to retain explicit per-subcase content instead of collapsing all modal case-control requests into one effective solve: the solver now honors subcase-level `METHOD`, `SPC`, optional thermal selectors, and `MODES` counts, while the JSON and HDF5 exports preserve those per-subcase modal groups alongside the historical combined top-level mode arrays for backward compatibility. The experimental SOL 106 branch now preserves its nonlinear content in two complementary ways: the aggregated `.JU.JSON` export keeps the usual static-style result arrays but also embeds per-subcase nonlinear diagnostics and a compact nonlinear solver summary, while the dedicated `.NONLINEAR.JSON` file stores per-subcase nonlinear histories together with the paired linear-solver diagnostics used inside the nonlinear correction loop. Those SOL 106 diagnostics include nonlinear load-step histories, per-step convergence records, residual norms before and after accepted steps, relative-change and normalized incremental-work measures, internal-force and geometric-force norms, the selected nonlinear method, residual model, and correction-tangent model, per-iteration correction-solver records, explicit line-search acceptance reasons, selected-trial indices, selection-policy labels, and reduction flags, accepted line-search scales, cumulative trial and backtrack counts, adaptive cutback attempt histories, adaptive next-step growth reasons, accepted versus requested step counts, explicit termination reasons, counts of nonlinear state evaluations, counts of accepted-state reuse, counts of step-growth events, summary counts of how often the accepted formal trial diverged from the minimum-merit or minimum-admissible trial references, maximum gaps between those trial choices, and summaries of the final geometric-stiffness update or formal shell tangent contribution. This is important because the formal branch can now be identified and audited directly from exported data rather than inferred from console logs. The buckling adjoint output now adds a second layer of reporting: per-design-variable path diagnostics list the target groups and identify which ones used full-model $\partial\mathbf{K}/\partial x$, full-model fixed-$\mathbf{u}$ $\partial\mathbf{K}_G/\partial x$, or the remaining mode-tracked complete-sensitivity fallback, while a compact summary block counts how many design variables and groups follow each route. This is an important practical improvement

because automated studies can now detect solver health and sensitivity-path cost from JSON output rather than only from console text. Recent parser/regression work also added native permanent-constraint coverage for `GRID`/`GRDSET` `PS` fields: a focused end-to-end rod regression now exercises both explicit `GRID` `PS` data and inherited `GRDSET` defaults through the direct BDF and raw-JSON routes, and the checked-in rod constraint case raw-JSON parity regression now asserts that its inherited `GRDSET` component string survives identically in both derived models. The checked-in solid-modal case solid-modal regression now does more than verify mass assembly: it explicitly guards the duplicate-public-ID case that appears in that deck, checks that the direct and raw-JSON paths both preserve the expected 4 `CHEXA` plus 1 `CPENTA` topology, rejects the old spurious near-zero mode by requiring the restored lateral first frequency to remain well above that former mechanism, and now also verifies that the exported per-subcase modal groups preserve the deck's requested `MODES` counts $(1, 3, 1, 3)$. The same test still solves the deck once as checked in and once after scaling the structural density by a factor of four in memory, and it asserts the expected $4\times$ increase in assembled modal mass together with the corresponding $2\times$ drop in the first natural frequency. This is still not a full Reference parity statement, but it is now a materially stronger guard against silently dropping solid contributions or overwriting colliding element IDs inside the current SOL 103 path.

6 Advanced Capabilities: Adjoint Analysis and Differentiable Infrastructure

6.1 Static adjoint sensitivities

For linear static analysis the repository implements a conventional discrete adjoint method. If the equilibrium equation is

$$\mathbf{K}(\mathbf{x})\mathbf{u} = \mathbf{F}(\mathbf{x}), \tag{7}$$

and a scalar response $g(\mathbf{u}, \mathbf{x})$ is sought, the total derivative can be written as

$$\frac{\mathrm{d}g}{\mathrm{d}x_i} = \frac{\partial g}{\partial x_i}\boldsymbol{\lambda}^{\mathsf{T}}\left(\frac{\partial \mathbf{F}}{\partial x_i} - \frac{\partial \mathbf{K}}{\partial x_i}\mathbf{u}\right), \tag{8}$$

with the adjoint vector obtained from

$$\mathbf{K}^{\mathsf{T}}\boldsymbol{\lambda} = \frac{\partial g}{\partial \mathbf{u}}. \tag{9}$$

Because the governing operator is symmetric for the supported linear structural cases, the adjoint system reuses the same stiffness operator structure as the forward problem.

The implementation in `src/solver/adjoint.jl` exposes several response functionals through a JSON configuration file placed next to the input deck. The supported checked-in response families are

- nodal displacement components,
- elemental von Mises stress responses,
- shell membrane resultants N_x, N_y, and N_{xy},
- shell bending moments M_x and M_y.

This response library is modest but practically useful. It covers compliance-like displacement targets, local stress interrogation, and laminate or panel force-resultant objectives that are common in sizing studies.

6.2 Buckling eigenvalue sensitivities

The buckling adjoint capability extends the same philosophy to eigenvalue responses. The source comments in `src/solver/adjoint_buckling.jl` state the implemented sensitivity in the form

$$\frac{\mathrm{d}\lambda}{\mathrm{d}x} = -\frac{1}{\boldsymbol{\phi}^{\mathsf{T}}\mathbf{K}_G\boldsymbol{\phi}}\left[\boldsymbol{\phi}^{\mathsf{T}}\frac{\partial\mathbf{K}}{\partial x}\boldsymbol{\phi} \;+\; \lambda\boldsymbol{\phi}^{\mathsf{T}}\left.\frac{\partial\mathbf{K}_G}{\partial x}\right|_{\mathbf{u}\,\text{fixed}}\boldsymbol{\phi} \;-\; \boldsymbol{\psi}^{\mathsf{T}}\frac{\partial\mathbf{K}}{\partial x}\mathbf{u}\right], \tag{10}$$

where the auxiliary vector $\boldsymbol{\psi}$ is obtained from a second linear solve driven by the displacement-dependence of the geometric stiffness.

This is a meaningful capability for a compact research code because buckling sensitivities are often more difficult to organize than static ones. Here they are integrated with the same derivative backend registry used elsewhere, so laminate, geometric, or material perturbations can be propagated into the buckling load factor without rewriting the eigensolver. In the present implementation, the shell-thickness contribution to $\partial\mathbf{K}_G/\partial h$ exploits the exact proportionality $\mathbf{K}_G/h$ on the standard shell path, while the newer specialized flat-curved, explicit covariant-operator, and compatible-membrane auto-covariant isotropic quadrilateral subsets use branch-aware local finite differences so that per-PID thickness contractions remain synchronized with the forward SOL 105 shell operator. For shell Poisson ratio, the standard and flat specialized shell branches still use the exact shell-level relation $\partial\boldsymbol{\sigma}/\partial\nu = (\partial\mathbf{D}/\partial\nu)\mathbf{B}\mathbf{u}$ together with the linearity of $\mathbf{K}_G$ in the membrane stress resultants, while the verified covariant-operator quadrilateral subset now evaluates the same fixed-$\mathbf{u}$ contraction through synchronized branch-aware local derivatives instead of a full-model finite-difference reassembly. Recent refinements reduce a different source of mismatch on quadrilateral-shell buckling decks. For flat orthotropic PCOMP `CQUAD4` elements, the adjoint RHS no longer assumes a generic quadrilateral geometric-stiffness operator only, but reconstructs the same material rotation, membrane-force recovery, and specialized operator selection (`DKMQ`, `DKQ`, or plate-like generic $\mathbf{K}_G$) used by the forward SOL 105 assembly. For flat isotropic `CQUAD4` elements, the adjoint now mirrors the solver's exact-membrane membrane-force recovery and the associated generic $\mathbf{K}_G$ operator settings, and the flat-isotropic $\partial\mathbf{K}_G/\partial\nu$ term is evaluated on that same branch-aware subset rather than through a purely centroidal approximation. A further refinement now covers three additional isotropic quadrilateral SOL 105 variants. The first is the flat-curved exact-membrane subset encountered on faceted cylinder panels, where the adjoint now follows the same geomnormal-frame, nodal-transform, curvature-aware membrane-force recovery, and exact-membrane branch selection used by the forward geometric-stiffness assembly. When the forward solve is configured to use the covariant shell-surface geometric-stiffness operator, the adjoint reproduces the same explicit covariant subset for both flat and curved geomnormal-frame isotropic quadrilateral shells. When the solver instead uses compatible membrane stress recovery together with the auto-covariant membrane selector, the adjoint now reproduces that verified isotropic auto-covariant membrane-recovery subset as well. In those specialized isotropic cases the buckling RHS is linearized with the active prestress-field selector frozen, including residual, Gauss-point, and blended sigma-field choices, whereas the shell $\partial\mathbf{K}_G/\partial\nu$ and $\partial\mathbf{K}_G/\partial h$ contractions are synchronized through matching branch-aware local derivatives on the presently verified subsets. The code now also carries the static and eigenvalue stiffness operators separately through the SOL 105 results, so full-model buckling derivatives can evaluate the $\boldsymbol{\phi}^{\mathsf{T}}\partial\mathbf{K}_{\text{eig}}/\partial x\,\boldsymbol{\phi}$ and $\boldsymbol{\psi}^{\mathsf{T}}\partial\mathbf{K}_{\text{static}}/\partial x\,\mathbf{u}$ terms on the correct operators,

caches those reassembled stiffness and geometric-stiffness derivatives across modes, and supplements the hardest exact-membrane groups with fixed-$\mathbf{u}$ full-model $\partial \mathbf{K}_G / \partial x$ contractions before applying the remaining workflow-level fallback. The same grouped buckling path now preserves the fixed-$\mathbf{u}$ geometric-stiffness terms for analytical non-shell grouped variables such as Young's modulus, which avoids small but systematic cancellation errors in low-signal eigenvalue sensitivities after the newer cache-partitioning refactors. For the exact-membrane curved-shell groups on which the fully local decomposition still proves brittle, the implementation therefore retains a narrower mode-tracked full-model finite-difference fallback for the complete group sensitivity, which is what closes the end-to-end mismatch on the small barrel and tapered-cylinder benchmark families. The solver now reports those local, cached full-model, and complete-sensitivity routes explicitly through per-design-variable path diagnostics and a compact path summary, so expensive branches remain auditable in large automated studies. This is a practical way to keep the adjoint consistent with the forward shell branch without introducing a second symbolic shell implementation.

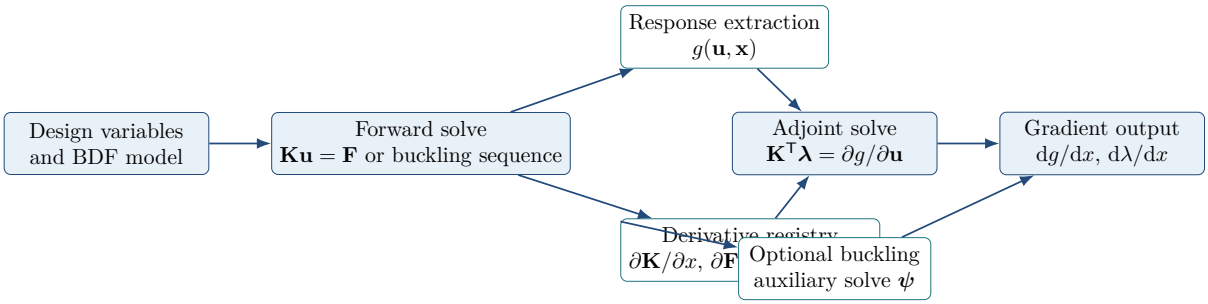


Figure 4: Adjoint workflow implemented by the repository. A forward solve produces the state and response quantities, the derivative registry supplies $\partial \mathbf{K}/\partial x$ or equivalent pseudo-load actions, and one or two adjoint solves convert those ingredients into gradients.

6.3 Derivative back-ends and partial automatic differentiation

One of the most interesting design choices appears in `src/solver/dKdx.jl`. Rather than hard-coding every sensitivity separately, the repository registers each design-variable type with a derivative back-end. Table 5 summarizes the variables explicitly visible in the checked-in code.

This table deserves careful interpretation. The repository is already *differentiable in practice* because gradients can be produced for several useful variables and responses. The present code now depends on `ForwardDiff` and uses it in real, if still limited, forward-mode AD paths: `bar_area` sensitivities are evaluated by differentiating a generic frame-element stiffness kernel with respect to area, `shell_thickness` now uses ForwardDiff on both triangular shells and the default quadrilateral shell operator exercised by the adjoint thickness routine, and `material_NU` uses the same strategy on the standard shell stiffness kernels while the buckling adjoint combines exact shell-level $\partial \mathbf{K}_G / \partial \nu$ contractions on the standard and flat specialized branches with synchronized branch-aware local contractions on the specialized covariant-operator subset. In parallel, the laminate-ply branch has advanced beyond pure finite differences: `pcomp_ply_thickness` and `pcomp_ply_angle` now differentiate the stored CLT laminate matrices directly, while retaining the older CLT finite-difference route as a compatibility fallback. The buckling adjoint has also narrowed several of its previous shell-operator gaps by adding branch-aware flat-PCOMP quadrilateral RHS evaluation, a matching flat-isotropic quadrilateral subset for both the buckling RHS and the flat-shell $\partial \mathbf{K}_G / \partial \nu$ contraction, a branch-aware flat-curved isotropic exact-membrane subset verified on a faceted cylinder-

Table 5: Design-variable types and derivative strategies exposed by the solver.

Design variable	**Derivative back-end**	**Interpretation in the current implementation**
`shell_thickness`	`:ad_forward`	ForwardDiff-backed thickness sensitivity on the shell elements used by the adjoint path, including `CTRIA3` and the default `CQUAD4` stiffness kernel.
`material_E`	`:analytical`	Analytical stiffness derivative with respect to Young's modulus.
`material_NU`	`:ad_forward` with solid fallback	ForwardDiff-backed Poisson-ratio sensitivity on the standard shell stiffness kernels, with the existing finite-difference solid-element path retained; on the buckling side the shell $\partial \mathbf{K}_G / \partial \nu$ contraction is evaluated exactly from $\partial \boldsymbol{\sigma} / \partial \nu$ on the standard and flat specialized shell paths and through synchronized branch-aware local contractions on the verified covariant subsets.
`bar_area`	`:ad_forward`	Cross-sectional sizing sensitivity for line elements through ForwardDiff acting on a generic frame-element stiffness kernel.
`pcomp_ply_thickness`	`:laminate_exact` with `:clt_fd` fallback	Exact derivatives of the stored CLT membrane, coupling, bending, and shear matrices with respect to a selected ply thickness, using stored ply material identifiers when needed.
`pcomp_ply_angle`	`:laminate_exact` with `:clt_fd` fallback	Exact derivatives of the laminate matrices with respect to ply angle, using the ply material data and ForwardDiff at the transformed-constitutive level.
`node_coord`	`:full_model_fd`	Global reassembly after geometric perturbation of a nodal coordinate.
`topology_density`	`:analytical`	Element-density-like scaling used in tests and topology-oriented sensitivity studies.
`AD override hook`	`:ad_forward`	Backend override path exposed per design variable so supported variables can request ForwardDiff explicitly while other variables continue to use FD or analytical forms.

panel deck for both the buckling RHS and the shell $\partial \mathbf{K}_G / \partial \nu$ contraction, branch-aware split-PID shell-thickness checks on that same flat-curved subset, an explicit covariant isotropic `CQUAD4` subset for the buckling RHS together with synchronized shell $\partial \mathbf{K}_G / \partial \nu$ and split-group shell-thickness evaluations on the corresponding covariant-operator subset, now including a curved geomnormal-frame saddle-shell branch and a split-MID Poisson-ratio regression on a multi-element deck, and a verified compatible-membrane auto-covariant isotropic membrane-recovery subset for the buckling RHS, the shell $\partial \mathbf{K}_G / \partial \nu$ contraction, and split-PID shell-thickness contractions. Another architectural refinement now separates the buckling $\mathbf{K}_{\text{eig}}$ and static $\mathbf{K}_{\text{static}}$ operators inside the full-model SOL 105 bilinear terms and supplements the hardest exact-membrane groups with fixed-$\mathbf{u}$ full-model $\partial \mathbf{K}_G / \partial x$ contractions before applying the remaining narrow mode-tracked complete-sensitivity fallback. That is a meaningful architectural step because it demonstrates that the derivative registry is not merely aspirational and that AD, exact laminate- or shell-level sensitivities, branch-aware adjoint linearizations, split buckling operators, and targeted workflow-level fallbacks can be enabled on several important sizing and stability paths. At the same time, coverage is still partial rather than universal. Nodal-coordinate, solid Poisson-ratio, and many specialized shell-operator branches beyond the flat orthotropic PCOMP, flat isotropic, flat-curved exact-membrane isotropic, verified covariant-operator isotropic, and the presently verified compatible-membrane auto-covariant isotropic quadrilateral buckling subsets continue to be handled analytically or by finite differences, so the most accurate description of the present repository is a solver with *mixed analytical, finite-difference, exact laminate- and shell-level, branch-aware adjoint, split-operator buckling reassembly, targeted workflow-level buckling fallbacks, and partial forward-mode AD backends.*

6.4 Sizing and optimization hooks

The repository also includes optimization drivers in `src/solver/optimize_thickness.jl`. Two objective families are implemented:

- minimum-compliance sizing optimization for SOL 101 using an optimality-criteria style update,
- buckling-factor maximization for SOL 105 using projected gradient updates.

The implementation can split shell properties into per-element property identifiers so that nominally shared thicknesses become independently designable, and it now applies the same idea to bar and beam properties so that per-element area variables can be introduced for `CBAR`/`CBEAM` sizing studies. In the current source tree these drivers do more than return a final design map: they record structured iteration objects containing objective value, relative change, mass, variable bounds, gradient norm, and embedded forward-solver diagnostics. They can also write JSON checkpoints at user-defined intervals, restart from a previous checkpoint, and accept explicit derivative-backend selections for both shell-thickness and bar-area variables. The current optimization payloads now also expose per-design-variable trajectories, active-constraint histories, per-constraint trajectory groups, and normalized routed-response margins so the same information survives into the exported `.OPTIMIZATION.JSON` and `.OPTIMIZATION.H5` files, while private cached solve state used internally by the routed search is intentionally stripped from those exported artifacts. On the routed static subset the implementation also now reuses the last inner forward solve when packaging the final optimization payload instead of repeating an equivalent final solve solely for export bookkeeping.

The parser and model builder now also recognize a narrow SOL-200-lite input subset consisting of DESVAR, DRESP1, DVPREL1, DVMREL1, DCONSTR, and DOPTPRM bulk-data entries together with case-control DESOBJ and DESSUB metadata. Those cards are normalized into an internal OPTIMIZATION dictionary attached to the model, with preserved objective sense modifiers such as DESOBJ(MIN). The normalized readiness metadata now distinguishes parser-level recognition from the narrower relation/response family set that the routed dispatcher actually executes, so parsed-but-unexecuted material relations such as DVMREL1 can be reported explicitly without pretending the corresponding route already exists. The current source tree now goes one step beyond parser-only recognition: a narrow execution path translates the subset that is equivalent to the existing sizing driver, namely compliance minimization or first-buckling maximization with a single upper-bound WEIGHT/MASS constraint and supported shell-thickness or bar-area DVPREL1 relations, into the restartable sizing optimizer already used elsewhere in the code. A second still narrower route now handles DESOBJ(MIN)=WEIGHT/MASS on the static subset. When the routed problem collapses to one grouped supported shell/bar variable and one or more upper-bound compliance or displacement constraints, JFEM performs an exact one-variable bracket-and-bisect search on the routed SOL 101 solve. When the routed problem contains multiple grouped supported shell/bar variables and one or more upper-bound compliance or displacement constraints, JFEM now performs a narrow outer mass-target bisection on the same routed SOL 101 subset, falls back to a direct feasible cleanup if the expected monotonicity over that bracket is not observed, and then applies a small feasible mass-polish pass driven by the routed static sensitivities. The inner reduction still stays deliberately limited: it solves either grouped static-response minimizations when only one routed bound is active or grouped static-constraint reductions with a smooth aggregate of the routed constraint set when several compliance/displacement bounds must be satisfied simultaneously, and the fallback and final polish never leave that same supported static shell/bar subset. On the routed subset the implementation now preserves tied DESVAR→DVPREL1 semantics exactly instead of expanding them into independent per-element variables: if one supported DESVAR drives multiple shell-thickness or bar-area relations, including mixed shell-plus-bar groupings in which the same DESVAR spans both PSHELL and PBARL properties, or a routed property is shared by multiple active elements, the translator still performs the internal per-element property split needed by the optimizer but keeps all split properties tied under one grouped design variable and initializes them from DESVAR.XINIT before the translated solve begins. The translated mass bookkeeping on these routes now also carries fixed shell/bar non-structural mass contributions from PSHELL.NSM, PBARL.NSM, and explicit PBAR.NSM, so the reported routed WEIGHT/MASS targets and final optimization payloads distinguish fixed mass offsets from the reducible shell-thickness or bar-area portion. The same routed path now also consumes DOPTPRM DELX as the default grouped move limit when a supported DESVAR leaves DELX blank. Those payloads now also record whether the final polish was attempted, whether the nonmonotone fallback was used, and how much mass the cleanup removed. Unsupported SOL 200 requests still fail explicitly rather than silently falling through to the SOL 101 path.

At the implementation level, restartability is enabled by preserving per-element optimization metadata in the internal model and by serializing iteration histories together with the current design field. Dedicated private checks exercise both the thickness-only and mixed shell/bar restart paths through the maintained code-development and validation-tool areas. A SOL-200-lite parser smoke test now verifies recognition of DESOBJ(MIN), DESVAR, DRESP1, DVPREL1, DVMREL1, DCONSTR, and DOPTPRM on a small shell deck, confirms that the normalized readiness metadata flags parsed-but-unexecuted DVMREL1 content explicitly, and confirms that unsupported requests still fail explicitly. Companion SOL-200-lite routed-workflow smokes now cover the original one-bar compliance case, now with a PBARL.NSM fixed-mass offset carried through the translated WEIGHT/MASS target, a paral-

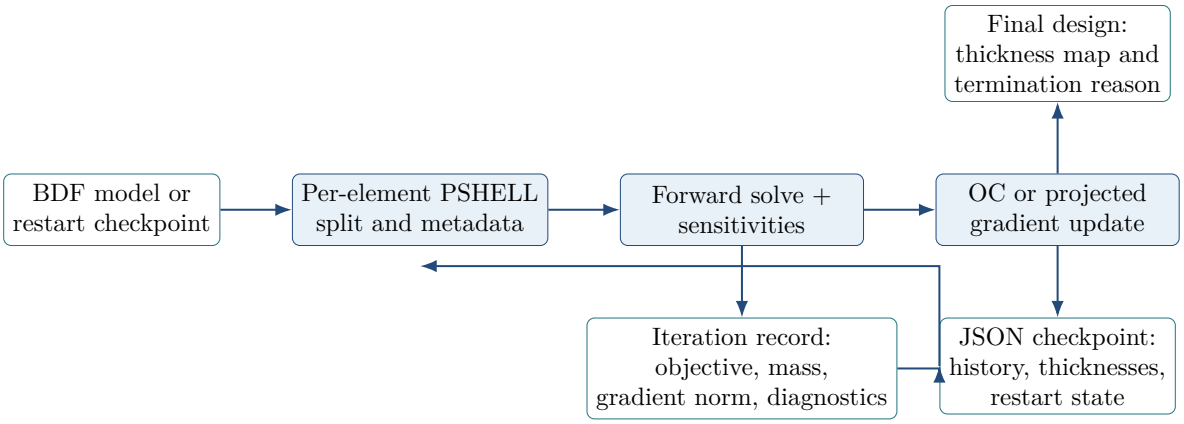


Figure 5: Optimization workflow now implemented in the repository. Shell thicknesses and, in the generalized sizing path, bar areas are created through per-element property splitting; each iteration records objective and solver-health data, and the process can be checkpointed and restarted from JSON.

lel one-bar explicit-PBAR compliance case that verifies the same routed bar-area path on normalized explicit section data and its corresponding explicit-PBAR.NSM bookkeeping, a matching one-bar explicit-PBAR exact static mass-minimization case with a routed displacement bound on that same normalized section family, grouped-bar compliance cases on both PBARL and explicit PBAR properties in which one DESVAR drives multiple routed bar-area relations used by several active elements, a dedicated one-bar compliance smoke in which DESVAR.DELX is blank so the routed move limit must fall back to DOPTPRM DELX, grouped exact-search mass-minimization cases with either displacement or compliance upper bounds, a mixed static-constraint case that enforces both bounds simultaneously on the same routed variable, an explicit-PBAR counterpart to that grouped mixed-constraint exact-search route, a two-group routed displacement-limited mass-minimization case with different bar stiffness groups and a nontrivial analytical optimum, the matching two-group explicit-PBAR variant of that routed mass-minimization case, a stricter two-group routed mass-minimization case with simultaneous mid-span and tip displacement bounds on that grouped bar subset, now also with the matching explicit-PBAR variant on the same routed multi-group multi-constraint bar path, a dedicated loose-tolerance two-group bar regression that proves the final feasible mass-polish stage can recover additional mass after the bracketed search, a companion forced-nonmonotone regression that now proves the route drops into its direct feasible fallback instead of erroring out when the bracket monotonicity assumption is violated, the corresponding grouped-PSHELL shell-thickness variant on a four-element membrane strip with the same two routed bounds, a mixed shell-plus-bar routed deck that now also checks fixed PSHELL.NSM and PBARL.NSM accounting while exercising PSHELL thickness and PBARL area groups together and preserving design-variable trajectories, active-constraint history, normalized routed-response margins, and the new mass-polish summary in export, a matching mixed shell-plus-explicit-PBAR routed deck on the same multi-group subset, a one-group mixed shell-plus-bar exact-search case in which a single DESVAR drives both PSHELL.T and PBARL.A, and now the matching one-group mixed shell-plus-explicit-PBAR exact-search case in which the same grouped route spans PSHELL.T and explicit PBAR.A while preserving the normalized bar fixed-mass bookkeeping through optimization and export: on the supported routed subset the solve preserves the grouped variable count, initializes the split properties from DESVAR.XINIT, reaches the translated mass or response limit, and writes the expected optimization or forward-analysis exports successfully.

7 Verification Status, Scope, and Current Limits

The repository contains both focused derivative checks and a broader benchmark corpus. On the sensitivity side, private code and validation tools compare adjoint gradients against finite differences for static displacement and stress responses, buckling load factors, laminate ply variables, nodal coordinates, and density-like topology variables. Recent additions include dedicated regressions for the exact shell-level buckling contractions $\partial \mathbf{K}_G/\partial h$ and the standard-shell $\partial \mathbf{K}_G/\partial \nu$ paths, direct finite-difference verification of PCOMP ply sensitivities using the laminate-exact back-end, a composite buckling mode-tracking smoke test based on MAC matching, focused finite-difference checks of the branch-aware flat-PCOMP and flat-isotropic buckling adjoint RHS on one-element decks, a flat-isotropic branch-specific finite-difference check for the buckling $\partial \mathbf{K}_G/\partial \nu$ contraction, full faceted-cylinder-panel finite-difference checks for the flat-curved isotropic exact-membrane buckling RHS and shell $\partial \mathbf{K}_G/\partial \nu$ contraction, split-PID finite-difference checks for the corresponding flat-curved shell-thickness contraction, dedicated finite-difference checks for the explicit covariant isotropic `CQUAD4` buckling RHS and synchronized shell $\partial \mathbf{K}_G/\partial \nu$ contraction on both the earlier one-element subset and the newer curved geomnormal-frame saddle-shell branch, new split-group regressions on a multi-element covariant saddle-shell deck for both the explicit-covariant shell-thickness contraction and the split-MID Poisson-ratio contraction, a dedicated curved-shell buckling-adjoint finite-difference regression that verifies the exact-membrane barrel and tapered-cylinder families against mode-tracked full-model reference sensitivities after the new split-$\mathbf{K}_{\mathrm{eig}}/\mathbf{K}_{\mathrm{static}}$ operator treatment, a tightened low-signal Young's-modulus buckling regression that now catches grouped analytical-path drift even when the absolute derivative is very small, new MAC-based curved-shell mode-tracking regressions spanning a small barrel exact-membrane deck, large barrel and dome explicit-covariant families, and a tapered-cylinder exact-membrane subset, new focused checks for the compatible-membrane auto-covariant isotropic `CQUAD4` membrane-recovery branch on the buckling RHS, on the shell $\partial \mathbf{K}_G/\partial \nu$ contraction, and on split-PID shell-thickness contractions, a smoke test for the residual-controlled experimental SOL 106 branch on the cantilever problem, a dedicated SOL 106 shell-panel regression on a flat `PSHELL` deck using the explicit `formal_shell_von_karman` branch, a new multi-element flat `PSHELL` regression on a 2×2 panel that combines the original in-plane shear loading with a small center transverse point load so the same formal branch sees both membrane/shear and bending response on one workflow, a companion stronger 2×2 flat-panel regression with a four-times larger center transverse load that now forces repeated cutback recovery but still reaches full load on the same formal branch, a converged scaled-load SOL 106 formal cantilever regression that now records a rejected full trial followed by Armijo acceptance of a backtracked half step while still preserving the later scanned trial states, a new focused SOL 106 selector regression on the same scaled formal cantilever family that proves a later residual-reducing trial is preferred over an earlier Armijo-only candidate, additional focused SOL 106 formal line-search diagnostics that force repeated backtracking and, separately, the explicit best-available fallback when every recorded trial remains rejected, a dedicated SOL 106 cutback-diagnostics regression that forces step reduction and verifies the recorded retry history, correction-partition reuse, accepted-state reuse, and explicit termination reasons, a new SOL 106 step-growth regression that confirms the solver can finish in fewer accepted steps than the nominal request when successive increments converge cleanly, paired SOL 106 export smoke tests on the lighter legacy secant cantilever workflow and on the scaled formal cantilever workflow, and now a stronger formal mixed-panel export smoke that preserves the broader incremental-work and tiny-step diagnostic contract, including explicit plateau summaries when they are activated and the corresponding zero-case invariants when the stronger case converges without a tiny-step plateau, together with the new run-level accepted-

versus-merit divergence summaries, in both the embedded `.JU.JSON` payload and the separate `.NONLINEAR.JSON` file. The nonlinear tests therefore span eleven complementary cases: the explicit formal flat-shell branch on a membrane-dominated panel, a broader multi-element flat-panel case with mixed membrane and bending response, a stronger recovery-driven multi-element flat-panel case on the same family, a converged formal flat-shell cantilever path with accepted backtracking, a scaled formal cantilever selector case that chooses the best residual-reduction trial rather than the first admissible one, a formal flat-shell cantilever path that exercises Armijo acceptance after multiple backtracks, the same formal cantilever family forced into the best-available fallback, the legacy geometric branch with the secant-style residual option on the lighter cantilever workflow, the legacy export path that preserves those histories without recomputing a surrogate postprocessing solve, the corresponding formal export path on the explicit shell branch, and the stronger formal panel export path that now carries the full incremental-work and accepted-versus-merit diagnostic contract, including explicit plateau summaries when present and the corresponding zero-case invariants when they are absent. This is important because it shows the differentiable and nonlinear features were not only coded but also cross-checked numerically at workflow level.

At the workflow level, the latest completed private broad-corpus summary is maintained under the comparison archive. It is not part of the public GitHub repository because the underlying comparison decks are private. The public repository instead ships the curated public validation suite under `validation/`. Table 6 preserves the private-campaign headline counts as an internal development record.

Table 6: Current broad-corpus validation summary shipped with the repository.

Category	**Count**	**Interpretation**
Total decks processed	3997	Aggregate validation corpus examined by the comparison script.
OK	2048	Decks completed successfully in the current recorded run.
Solver error	973	Decks reached nominally supported workflows but failed numerically or procedurally.
Unsupported solution sequence	976	Decks requested solution families outside the current broad-corpus scope.
Effective deck fragment	267	Standalone INCLUDE fragments or otherwise incomplete paired decks, separated from true solver failures in the regenerated summary.

Two cautions are needed when interpreting those numbers. First, the broad-corpus summary now reports both raw and effective status counts: the raw supported-workflow completion is $2048/3021 \approx 67.8\%$, while the effective completion rises to $2048/2754 \approx 74.4\%$ after the runner separates 267 standalone deck fragments from genuine supported-workflow failures. Second, a successful deck count is not equivalent to feature parity with commercial Reference; it indicates useful breadth of coverage for the solver's present scope, not completeness across the full Reference language.

The main current limitations are therefore clear:

- the nonlinear SOL 106 path is currently marked experimental and now splits into a legacy geometric branch and an explicit formal flat-shell von Kármán branch; the formal route is

presently limited to flat coplanar `CTRIA3`/`CQUAD4` `PSHELL` cases, the legacy branch still uses residual-controlled correction iterations with a linear-plus-geometric tangent and configurable tangent-style or secant-style residual models, and neither branch yet constitutes a full Newton-consistent large-displacement formulation for the general shell and solid model space,

- higher-order shell cards are handled by subdivision rather than native high-order formulations,
- some sensitivities rely on localized finite differences instead of exact analytical or AD-derived gradients,
- the forward AD back-end is only partially activated, currently covering line-element area sizing, the standard `CTRIA3`/`CQUAD4` shell-thickness kernels, and shell Poisson-ratio derivatives rather than the full shell and laminate operator space,
- the buckling adjoint now mirrors the forward specialized branch logic for useful flat orthotropic PCOMP and flat isotropic quadrilateral subsets, for a verified flat-curved isotropic exact-membrane cylinder-panel subset and additional barrel and tapered-cylinder decks, for a verified explicit covariant isotropic `CQUAD4` covariant-operator subset that now includes a curved geomnormal-frame saddle-shell branch, split-group $\partial \mathbf{K}_G/\partial h$ and $\partial \mathbf{K}_G/\partial \nu$ checks, and broader barrel and dome mode-tracked families, and for a verified compatible-membrane auto-covariant isotropic membrane-recovery subset; the full-model SOL 105 path now distinguishes $\mathbf{K}_{\text{eig}}$ from $\mathbf{K}_{\text{static}}$ and adds fixed-$\mathbf{u}$ full-model $\partial \mathbf{K}_G/\partial x$ contractions on the hardest exact-membrane groups, but those same groups still fall back to mode-tracked full-model finite differences for the complete buckling sensitivity and many other specialized SOL 105 operator branches are still approximated more coarsely than the forward assembly,
- optimization now spans per-element shell thickness and bar area, and the parser can now build a normalized internal problem definition from a small SOL-200-lite subset of design-variable, response, relation, constraint, and optimizer-parameter cards; a narrow dispatcher now routes compliance-minimization or first-buckling-maximization problems with a single upper-bound mass constraint onto the existing sizing driver and also supports static mass-minimization on a tighter routed subset, using an exact one-group bracket-and-bisect search for one grouped shell/bar variable with one or more upper-bound compliance or displacement constraints and a multi-group routed search when several grouped shell/bar variables remain active with one or more such upper-bound routed constraints; those routes preserve grouped supported `DESVAR`→`DVPREL1` shell/bar relations across multiple active properties and elements and initialize the routed shell/bar properties from `DESVAR.XINIT`; even with those additions the implemented design space remains well short of broad stress-, displacement-, buckling-, and manufacturing-constrained optimization.

8 Conclusions

The Julia solver documented here is best understood as a compact research and engineering code with a BDF-compatible front-end, not as a toy parser and not as an attempt to replicate every industrial Reference feature. Within its declared scope it is technically substantial: it parses realistic BDF decks, builds a JSON-serializable model representation, solves static, modal, buckling, and experimental nonlinear problems on sparse operators, exports results to visualization and machine-readable formats, and supports gradient-based studies through adjoint sensitivities and restartable sizing loops.

Its most distinctive contribution is architectural. The repository separates parsing, solving, export, and differentiation cleanly enough that each part can evolve independently. Composite laminate reduction, rigid-element elimination, shell-focused buckling treatment, exact laminate and shell derivative hooks, branch-aware flat-PCOMP, flat-isotropic, flat-curved exact-membrane isotropic, explicit covariant-isotropic, and compatible-membrane auto-covariant buckling adjoint linearization, structured solver diagnostics, restartable optimization, and a derivative registry that now mixes analytical, finite-difference, exact laminate-level, synchronized branch-aware local covariant-operator buckling contractions, split static/eigen full-model SOL 105 bilinear forms with cached stiffness and geometric-stiffness derivative reassemblies, fixed-$\mathbf{u}$ full-model $\partial \mathbf{K}_G / \partial x$ contractions on the hardest exact-membrane groups, targeted mode-tracked full-model buckling fallbacks where those groups still need them, residual-controlled experimental SOL 106 correction iterations, an explicit formal flat-shell von Kármán internal-force branch with a consistent tangent on the currently supported subset, current-state partition reuse, accepted-state reuse, explicit termination diagnostics, direct accepted-state postprocessing, adaptive cutback/growth histories, and partial ForwardDiff back-ends all point to a code base designed for extension. The natural next step is to broaden the existing `:ad_forward`, exact-derivative, and branch-aware buckling coverage further into the remaining specialized quadrilateral-shell operators, broader auto-covariant and curved-shell branches, geometric derivatives, solid-material derivatives, and the experimental nonlinear path. Even before those steps, however, the present implementation already forms a credible platform for open structural analysis, method development, and gradient-informed design experiments.

A User's Guide

This appendix is intentionally practical. It is meant to help a new user go from a BDF deck to a solved and exported result with as little ambiguity as possible. The examples are taken from the checked-in repository decks and smoke tests, with short omissions marked by ellipses where repeated nodes or elements are not important to the pattern.

A.1 Command-line workflow

After the one-click installation step, the most direct way to run the solver is the repository-root launcher `jfem` on Linux/macOS or `jfem.cmd` on Windows. For lower-level automation, the same public runner is available at `utils/julia_tools/run_bdf.jl`. From the public repository root, the following commands are representative:

Listing 2: Typical command-line invocations from the repository root.

```
# Windows
.\jfem.cmd validation\cases\mystran_xref\sol101_simple.bdf runs\sol101_simple

# Linux/macOS
./jfem validation/cases/crm/wingbox_modal_with_props.bdf runs/crm_modal

# Explicit Julia entry point used by scripts and CI-style workflows
julia --project=. utils/julia_tools/run_bdf.jl \
  validation/cases/classical/timoshenko_plate_buckling.bdf runs/plate_buckling
```

These examples use only public validation decks. In each case the solver follows the same internal

sequence: parse the BDF into a model dictionary, solve the requested `SOL` path, and export machine-readable and visualization files to the requested run directory.

A.2 Programmatic workflow

For scripting, testing, or custom studies, the repository also exposes a clean parse–solve–export API through `JFEM/src/main.jl` and `JFEM/src/JFEMSolver.jl`. A minimal scripted usage pattern is shown in Listing 3.

Listing 3: Minimal programmatic workflow through the public API.

```
const JFEM_ROOT = "C:/path/to/repository/JFEM"
include(joinpath(JFEM_ROOT, "src", "main.jl"))

bdf_path = joinpath(JFEM_ROOT, "validation", "cases", "mystran_xref",
                    "sol101_simple.bdf")
output_dir = joinpath(JFEM_ROOT, "runs", "manual_run")
mkpath(output_dir)

model = bdf_to_model(bdf_path)
results = solve_model(model)
export_results(results, bdf_path, output_dir)
```

This API layer is particularly useful for the experimental nonlinear and optimization workflows because the model dictionary can be modified before solution. For example, a test can parse a baseline linear shell deck, change `model["SOL"]`, add nonlinear `PARAM_*` entries, and then solve the modified problem without writing a new intermediate deck to disk.

A.3 Choosing a solution sequence

Table 7 summarizes the most important input additions for each currently supported forward route.

A.4 Practical run order

In day-to-day use, the safest workflow is usually

1. start with a `SOL 101` deck and verify mesh, loads, and constraints,
2. add `METHOD`/`EIGRL` for modal work or a preload plus buckling subcase for `SOL 105`,
3. enable `SOL 106` only after the linear baseline is behaving as expected,
4. inspect the exported `.JU.JSON`, `.JU.H5`, `.NONLINEAR.JSON`, or `.NONLINEAR.H5` files when solver diagnostics matter,
5. use the VTK or JFEM exports for geometry- and field-level inspection.

For the current experimental nonlinear branch it is especially useful to retain both the aggregated and dedicated nonlinear exports, because the aggregated files preserve the static-style result arrays while the dedicated nonlinear files expose step histories, iteration diagnostics, accepted trial indices, cutbacks, and termination reasons directly.

Table 7: Practical input checklist for the supported solution sequences.

Route	Key deck ingredients	Typical exported files
SOL 101	SOL 101, at least one subcase with LOAD and SPC, structural bulk-data cards such as GRID, element, property, and material entries.	.JU.JSON, .JU.H5, .vtu, .jfem, .CARDS.JSON.
SOL 103	SOL 103, modal case-control METHOD = sid, and a bulk-data EIGRL card with the same SID. Mass-producing cards such as shell, bar, rod, or concentrated-mass entries must be present.	.BUCKLING.JSON tagged as modal, .MODES.H5, .vtu, .jfem.
SOL 105	A static preload subcase, a buckling subcase with STATSUB = preload_sid, and a modal extraction control such as METHOD = sid with matching EIGRL.	.BUCKLING.JSON, .BUCKLING.H5, .vtu, .jfem.
SOL 106	SOL 106, at least one static-style subcase with LOAD and SPC, plus nonlinear control through bulk-data PARAM cards such as NLLOADSTEPS, NLMAXITER, NLTOL, and optionally NLMETHOD.	.JU.JSON, .JU.H5, .NONLINEAR.JSON, .NONLINEAR.H5, .vtu, .jfem.
SOL 200-lite	SOL 200, case-control DESOBJ and DESSUB, plus the narrow supported subset of DESVAR, DRESP1, DVPREL1, DVMREL1, DCONSTR, and DOPTPRM.	.OPTIMIZATION.JSON, .OPTIMIZATION.H5, and the corresponding final routed forward-analysis exports.

B Input Deck Excerpts by Solution Sequence

The following excerpts are not intended to be exhaustive models. Instead they are compact patterns that show the solution-specific control lines together with the structural cards that normally accompany them.

B.1 Static analysis (`SOL 101`)

Listing 4 is adapted from the cantilever shell benchmark used throughout the repository smoke tests.

Listing 4: Compact static shell deck for `SOL 101`, adapted from the repository cantilever benchmark.

```
$ Cantilever plate excerpt
SOL 101
CEND
TITLE = CANTILEVER 4x8
SUBCASE 1
  LOAD = 1
  SPC = 1
  DISPLACEMENT = ALL
BEGIN BULK
MAT1    1       2.1+11          0.3     7850.0
PSHELL  1       1       0.005    1               1
GRID    1               0.0     0.0     0.0
GRID    2               0.25    0.0     0.0
GRID    6               0.0     0.25    0.0
GRID    7               0.25    0.25    0.0
CQUAD4  1       1       1       2       7       6
...
SPC1    1       123456  1       THRU    5
FORCE   1       41      0       1.0     0.0     0.0     -62.5
FORCE   1       42      0       1.0     0.0     0.0     -125.0
...
ENDDATA
```

B.2 Normal modes (`SOL 103`)

The modal route uses the same structural bulk data as the static cantilever, but adds modal case-control and a matching `EIGRL` entry. This is the pattern used by the dedicated HDF5 modal export smoke test.

Listing 5: Minimal modal pattern for `SOL 103`; adapted from the cantilever shell deck with modal controls added.

```
SOL 103
CEND
TITLE = CANTILEVER MODES
SUBCASE 1
  SPC = 1
  METHOD = 1
  DISPLACEMENT(SORT1,REAL) = ALL
  VECTOR(SORT1,REAL) = ALL
BEGIN BULK
```

```
EIGRL,1,0.0,1.0E12,3
MAT1,1,2.1E11,,0.30,7850.0
PSHELL,1,1,0.005
GRID,1,,0.0,0.0,0.0
GRID,2,,0.25,0.0,0.0
...
CQUAD4,1,1,1,2,7,6
...
SPC1,1,123456,1,THRU,5
ENDDATA
```

B.3 Linear buckling (SOL 105)

Listing 6 is a shortened but faithful excerpt from the flat isotropic buckling benchmark (uniform shear loading). The key idea is the two-subcase structure: one static preload solve followed by one buckling solve that references the preload through STATSUB.

Listing 6: Buckling deck excerpt for SOL 105, adapted from the repository flat isotropic buckling benchmark.

```
SOL 105
CEND
TITLE = kgid_q1_flat_pshell_iso_nxy_t000_modes
SUBCASE 1
  LABEL = Static
  SPC = 1
  LOAD = 1
SUBCASE 2
  LABEL = Buckling
  SPC = 1
  METHOD = 10
  STATSUB = 1
  DISPLACEMENT(SORT1,REAL) = ALL
  VECTOR(SORT1,REAL) = ALL
BEGIN BULK
PARAM   POST    -1
EIGRL   10      0.0     1.0+10  8
GRID    1                 0.0000  0.0000  0.0000
GRID    2                 1.0000  0.0000  0.0000
GRID    3                 0.0000  1.0000  0.0000
GRID    4                 1.0000  1.0000  0.0000
CQUAD4  1       1       1       2       4       3       0.0
MAT1    1       70000.          0.3
PSHELL  1       1       0.01    1
SPC1    1       3       1       2       3       4
...
FORCE   1       2       0        50.0000  0.0000  1.0000  0.0000
...
ENDDATA
```

B.4 Experimental nonlinear static (SOL 106)

The checked-in nonlinear smoke tests often start from a linear shell deck and then inject nonlinear controls through the model dictionary. In deck form, those same controls can be represented by

`PARAM` cards, because the model builder stores them internally as `PARAM_*` entries. Listing 7 shows the pattern.

Listing 7: Nonlinear control pattern for `SOL 106`. The structural bulk data can be reused from the cantilever shell deck.

```
SOL 106
CEND
TITLE = NONLINEAR CANTILEVER
SUBCASE 1
  LOAD = 1
  SPC = 1
BEGIN BULK
PARAM,NLLOADSTEPS,2
PARAM,NLMAXITER,4
PARAM,NLTOL,1.0E-5
PARAM,NLRESTOL,1.0E-4
PARAM,NLRELAX,1.0
PARAM,NLRESMODEL,SECANT_GEOMETRIC
PARAM,NLLSMAX,3
PARAM,NLLSREDUCE,0.5
PARAM,NLCUTMAX,2
PARAM,NLCUTREDUCE,0.5
PARAM,NLSTEPGROW,1.25
$ Optional formal flat-shell branch on the currently supported subset:
$ PARAM,NLMETHOD,FORMAL_SHELL_VON_KARMAN
MAT1,1,2.1E11,,0.30,7850.0
PSHELL,1,1,0.005
...
CQUAD4,1,1,1,2,7,6
SPC1,1,123456,1,THRU,5
FORCE,1,41,0,1.0,0.0,0.0,-62.5
...
ENDDATA
```

B.5 Narrow optimization route (`SOL 200-lite`)

The current optimization path is intentionally narrow, but the appendix is a good place to show its input style explicitly. Today the routed subset covers compliance minimization or first-buckling maximization with one upper-bound mass constraint, plus a smaller static mass-minimization branch with either one grouped shell/bar variable and one or more upper-bound compliance or displacement constraints or, on a tighter subset, multiple grouped shell/bar variables with one or more such upper-bound compliance or displacement constraints. Listing 8 is adapted from the end-to-end compliance-routing smoke test.

Listing 8: Compact `SOL 200-lite` deck excerpt adapted from the compliance-dispatch smoke test.

```
SOL 200
CEND
DESOBJ(MIN) = 301
DESSUB = 1
SUBCASE 1
  SPC = 1
  LOAD = 1
BEGIN BULK
GRID,1,,0.,0.,0.
```

```
GRID,2,,1.,0.,0.
CBAR,1,10,1,2,0.,0.,1.
PBARL,10,20,,ROD,0.1,5.0E-03
MAT1,20,1.0E7,,0.30,1.0
FORCE,1,2,,1.0,1.,0.,0.
SPC1,1,123456,1
SPC1,1,23456,2
DESVAR,1,A001,3.14159E-02,2.0E-02,5.0E-02,2.0E-01
DVPREL1,101,PBARL,10,A,2.0E-02,5.0E-02,0.0,1,1.0
DRESP1,301,COMPAX,COMP
DRESP1,401,WEIGHT,WEIGHT
DCONSTR,501,401,,2.82743E-02
DOPTPRM,DESMAX,3,CONV1,1.0E-12
ENDDATA
```

C Representative Card Excerpts

The final appendix section collects individual card patterns that appear repeatedly in the repository validation decks. The important point is that the parser accepts both traditional fixed-field formatting and free-field comma-separated syntax, so it is normal for a working model to contain both styles.

C.1 Shell, laminate, and material cards

Listing 9 groups a few shell and laminate cards that are particularly useful when building shell benchmarks or laminate studies.

Listing 9: Representative shell and laminate cards adapted from the cantilever and composite validation decks.

```
$ Geometry and shell topology
GRID    1               2.0     1.0     0.0             3456
GRID    2               2.0     0.0     0.0             3456
GRID    3               1.0     1.0     0.0             3456
GRID    4               1.0     0.0     0.0             3456
CQUAD4  1       100     4       2       1       3
CTRIA3,2,10,1,2,3

$ Isotropic shell property and material
PSHELL  10      20      0.01    20
MAT1    20      7.0E10          0.3     2700.0

$ Composite laminate property and orthotropic material
PCOMP   100                     10000.  HILL                            +DEF
+DEF    120     .0333   45.     YES     120     .0333   0.0     YES     +GHI
+GHI    120     .0333   45.     YES
MAT8    120     1.0E7   1.0E6   0.3     4.0E6   4.0E6   4.0E6           +JKL
+JKL                            2.E4            2.E3            1.4E4   +MNO
+MNO    1.E-4
```

C.2 Constraints, loads, mass, line elements, and optimization cards

Listing 10 collects several other cards that appear in the repository corpus: rigid-element constraints, distributed or nodal loading, concentrated mass, bar properties, modal extraction, and the narrow optimization control cards.

Listing 10: Representative rigid, load, mass, line-element, modal, and optimization cards adapted from the validation decks and smoke tests.

```
$ Rigid interpolation and concentrated mass
RBE2   ,2000  ,1000  ,123   ,10000 ,10010 ,10020 ,10030 ,10040 ,RBE21
+BE21  ,10002 ,10012 ,10022 ,10032 ,10042 ,10001 ,10041
CONM2   101     1               4.0

$ Point and surface loading
FORCE   200     202     1       2.793-3 1.      0.      0.
PLOAD4  300     301     1.                              301     304     +PL4300

$ Bar element and property
CBAR    101     2       19      20      0.0     0.0     1.0
PBARL,2,1,,T
,0.02,0.03,0.003,0.003

$ Modal extraction
EIGRL   10      0.0     50.0

$ Narrow SOL 200-lite control cards
DESVAR,1,T001,0.10,0.01,0.50,0.05
DVPREL1,101,PSHELL,10,T,0.01,0.50,0.0,1,1.0
DVMREL1,201,MAT1,20,E,1.0E6,2.0E7,0.0,2,1.0
DRESP1,301,WEIGHT,WEIGHT
DRESP1,401,UZTIP,DISP,,,2,3
DCONSTR,501,401,,0.02
DOPTPRM,DESMAX,25,CONV1,1.0E-4
```

D Supported Bulk-Data Cards and Field Usage

This appendix is intended as a code-level reference rather than a generic Reference summary. The tables below list the bulk-data cards that the current implementation parses, normalizes, or uses directly in assembly and solution. The field lists show the entries that the present code actually reads and stores; officially defined Reference fields that are currently ignored are omitted here on purpose.

For this appendix, the status labels mean:

- **Direct**: the card participates directly in the current model, assembly, loads, solve, or export path.
- **Normalized**: the card is accepted but converted into another internal representation before use.
- **Limited**: only a subset of the official semantics is currently interpreted.

The appendix focuses on bulk-data cards. Executive and case-control items such as SOL, SUBCASE, LOAD, SPC, MPC, METHOD, STATSUB, DESOBJ, and DESSUB are discussed earlier in the manuscript and in the first appendix. Large-field forms such as PBAR* and PBEAM* are accepted through the same parser path as their small-field counterparts.

D.1 Geometry and coordinate-definition cards

Table 8: Geometry and coordinate-definition cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
GRID	Direct	ID, CP, X1, X2, X3, CD, PS, SEID	Node identifier; analysis coordinates in coordinate system CP; displacement output/recovery system CD; permanent component constraints through PS; and superelement identifier SEID. Blank CP/CD/PS/SEID fields can inherit defaults from GRDSET.
GRDSET	Normalized	CP, CD, PS, SEID	Default grid data. The current implementation applies these defaults while constructing the derived GRID dictionary; the inherited PS component string then participates directly in boundary-condition partitioning and solver diagnostics.
CORD2R	Direct	CID, RID, A1-A3, B1-B3, C1-C3	Rectangular coordinate-system definition by origin point A, axial point B, and orientation point C, optionally referenced to RID.
CORD2C	Direct	CID, RID, A1-A3, B1-B3, C1-C3	Cylindrical coordinate-system definition. The same geometric fields are read and tagged internally as cylindrical before grid transformation.
CORD2S	Direct	CID, RID, A1-A3, B1-B3, C1-C3	Spherical coordinate-system definition. The solver stores the same defining points and applies spherical-to-Cartesian conversion during geometry transformation.
CORD1R	Limited	CID, fields 4-12 as numeric data	Recognized and passed through the same point-based extractor used for CORD2R. This is narrower than full Reference CORD1R semantics, so only simple numerically equivalent use should be considered safe at present.

Table 9: Shell, line, spring, and bushing element cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
CTRIA3	Direct	EID, PID, G1, G2, G3, THETA/MCID	Three-node shell element with property link PID; the optional orientation field is stored as THETA.
CTRIA6	Normalized	EID, PID, G1-G6, THETA/MCID	Six-node shell element accepted by the parser and expanded into lower-order sub-elements during assembly.
CQUAD4	Direct	EID, PID, G1, G2, G3, G4, THETA/MCID	Four-node shell element with stored orientation angle or material coordinate selector.
CQUAD8	Normalized	EID, PID, G1-G8, THETA/MCID	Eight-node shell element accepted by the parser and subdivided into lower-order shell pieces in assembly.
CSHEAR	Direct	EID, PID, G1, G2, G3, G4, THETA/MCID	Shear-panel-style four-node shell entry collected through the shell extractor and paired with PSHEAR.
CBAR	Direct	EID, PID, GA, GB, X1/X2/X3 or G0, PA, PB, WA, WB	Two-node bar with either an orientation vector or reference grid, optional pin flags, and end offsets.
CBEAM	Direct	EID, PID, GA, GB, X1/X2/X3 or G0, WA, WB	Two-node beam with orientation data and end offsets; the current property treatment uses the parsed section data and averaged beam-section quantities where needed.
CROD	Direct	EID, PID, GA, GB	Property-based rod element.
CONROD	Direct	EID, GA, GB, MID, A, J, C, NSM	Rod element with inline material and section data rather than a separate property card.
CELAS1	Direct	EID, PID, G1, C1, G2, C2	Scalar spring referencing PELAS.
CELAS2	Direct	EID, K, G1, C1, G2, C2	Scalar spring with inline stiffness value.
CBUSH	Limited	EID, PID, GA, GB, CID	Six-degree-of-freedom bushing element. The current parser stores the connectivity, property link, and coordinate-system reference, while only a limited orientation subset is preserved explicitly.

Table 10: Mass and solid-element cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
CONM2	Direct	EID, GID, CID, M, X1, X2, X3, I11, I21, I22, I31, I32, I33	Concentrated mass attached to a grid, with optional offset and inertia terms.
CONM1	Limited	EID, GID, CID, upper-triangle mass terms	Full matrix form is recognized; the current model stores the diagonalized mass view used by the present solver paths together with the nominal mass value.
CMASS1	Direct	EID, PID, G1, C1, G2, C2	Scalar mass referencing PMASS.
CMASS2	Direct	EID, M, G1, C1, G2, C2	Scalar mass with inline value.
CTETRA	Direct	EID, PID, G1-G4	Four-node tetrahedral solid.
CHEXA	Direct	EID, PID, G1-G8	Eight-node hexahedral solid.
CPENTA	Direct	EID, PID, G1-G6	Six-node pentahedral solid.

D.2 Element cards

D.3 Property and material cards

D.4 Constraint and interpolation cards

D.5 Load, thermal, matrix-input, and analysis-control cards

D.6 Optimization cards

Table 11: Property cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
PSHELL	Direct	PID, MID1, T, MID2, 12I/T^3, MID3, TS/T, NSM	Shell property with membrane, bending, and transverse-shear material links, thickness, bending ratio, shear factor, and nonstructural mass per area.
PSHEAR	Direct	PID, MID, T, NSM	Shear-panel property paired with CSHEAR.
PBARL	Direct	PID, MID, TYPE, DIM..., NSM	Parametric bar section. The parser reads the section type, dimensions, and the supported trailing nonstructural-mass field, then converts them into area, moments of inertia, torsion constant, stress-recovery coordinates, approximate shear factors, and fixed mass metadata used by gravity and routed optimization bookkeeping.
PBAR	Direct	PID, MID, A, I1, I2, J, NSM, C1-F2, K1, K2, I12	Explicit bar section with section properties, stress-recovery coordinates, and shear factors. Large-field PBAR* follows the same path.
PBEAM	Limited	PID, MID, station A and B section data, C1-F2, K1, K2	Beam section with stationwise data. The current implementation reads the end-section information and uses averaged section properties for the present beam solver path. Large-field PBEAM* follows the same path.
PBEAML	Normalized	PID, MID, TYPE, DIM...	Treated through the same section-type and dimension extractor as PBARL. This is most reliable for the constant-section families currently exercised in the repository tests.
PROD	Direct	PID, MID, A, J, C, NSM	Rod property with area, torsion constant, stress coefficient, and nonstructural mass.
PCOMP	Direct	PID, Z0, NSM, LAM, (MID, T, THETA, SOUT) ply tuples	Composite laminate property. The current model builder converts the ply stack into CLT membrane, coupling, bending, and shear matrices and retains ply-level metadata.
PELAS	Direct	PID, K, GE	Scalar spring property with stiffness and damping coefficient.
PBUSH	Limited	PID, K1-K6	Bushing property. The current parser stores the six translational/rotational stiffness entries and ignores richer damping or nonlinear options.
PMASS	Direct	PID, M	Scalar mass property used by CMASS1.
PSOLID	Limited	PID, MID, CORDM, IN, STRESS, ISOP	Solid property with material link and optional formulation flags. The current solver uses the material link directly in static, modal, buckling, and current nonlinear solid workflows, while the remaining formulation flags are mostly preserved for compatibility rather than exhaustively interpreted.

Table 12: Material cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
MAT1	Direct	MID, E, G, NU, RHO, ALPHA, TREF	Isotropic material. If one of E, G, or NU is omitted, the parser reconstructs it from the other two when possible.
MAT2	Limited	MID, G11, G12, G13, G22, G23, G33, RHO	Anisotropic two-dimensional material. The parser also derives an equivalent compatibility E/G/NU view for workflows that expect scalar material parameters.
MAT8	Direct	MID, E1, E2, NU12, G12, G1Z, G2Z, RHO	Orthotropic shell material used directly in laminate and orthotropic shell workflows.
MATT1	Direct	MID, E(T), G(T), NU(T), RHO(T), ALPHA(T)	Temperature-dependent continuation of MAT1. The current implementation stores the referenced TABLEM1 ids and applies them on the isotropic MAT1-backed line, shell, and solid stiffness paths, on the corresponding thermal-prestress paths, and on the line, shell, and solid mass path used by SOL 103.
TABLEM1	Direct	TID, repeated (X, Y) pairs, ENDT	Tabulated scalar function used by MATT1. The parser reads the flattened numeric pairs across continuation fields and linearly interpolates them at the active subcase temperature.

Table 13: Constraint and rigid-interpolation cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
SPC1	Direct	SID, C, G-list/THRU	Constraint set identifier, component string, and expanded node list.
SPC	Direct	SID, G1, C1, D1, G2, C2, D2	Explicit enforced single-point constraints; the current parser supports up to two grid/component/value triplets per card line.
SPCADD	Direct	SID, subordinate set ids	Set-combination card for SPC sets.
MPC	Direct	SID, repeated (G, C, A) terms	Multi-point constraint coefficients used directly in the elimination path.
MPCADD	Direct	SID, subordinate set ids	Set-combination card for MPC sets.
RBE2	Direct	EID, GN, CM, GM...	Rigid-body interpolation with independent grid GN, constrained component mask CM, and dependent grids GM.
RBAR	Normalized	EID, GA, GB, CNA, CNB, CMA, CMB	Accepted and converted into an RBE2-equivalent internal constraint according to the present implementation's master/slave logic.
RBE1	Direct	EID, independent (G,C) pairs, UM, dependent (G,C) pairs	Rigid element with explicitly listed independent and dependent degree-of-freedom pairs.
RBE3	Direct	EID, REFGRID, REFC, weight groups	Interpolation element with a reference grid/component pair and one or more weighted dependent grid groups.
RSPLINE	Limited	EID, D/L, ordered grid/component sequence	Spline-style interpolation card. The current parser expands the independent grid chain and dependent component list used by the present constraint treatment.

Table 14: Mechanical, thermal, and load-combination cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
FORCE	Direct	SID, G, CID, F, N1, N2, N3	Concentrated force magnitude and direction at grid G.
MOMENT	Direct	SID, G, CID, M, N1, N2, N3	Concentrated moment magnitude and direction at grid G.
PLOAD4	Direct	SID, EID, P, optional G1, G3, CID, N1, N2, N3	Pressure load on a shell or solid face. The current parser stores the target element, pressure value, optional solid-face identifiers, and optional load-direction vector.
PLOAD2	Normalized	SID, P, EID...	Uniform element pressure card accepted and converted into the same internal pressure list used by PLOAD4.
PLOAD	Direct	SID, P, G1, G2, G3, G4	Grid-defined pressure patch with three or four corner nodes.
PLOAD1	Direct	SID, EID, TYPE, SCALE, X1, P1, X2, P2	Distributed load along a bar or beam element; the parser accepts both numeric and mnemonic load-type selectors.
GRAV	Direct	SID, CID, A, N1, N2, N3	Gravity vector definition with scale factor and direction.
RFORCE	Direct	SID, G, CID, A, R1, R2, R3, METHOD	Rotational body-force input used for centrifugal loading.
LOAD	Direct	SID, S, repeated (Si, Li) pairs	Linear combination of subordinate load sets.
TEMP	Direct	SID, repeated (Gi, Ti) pairs over the card and continuations	Nodal temperature map for thermal loading. The same map is also used when a subcase requests TEMP(LOAD) for temperature-dependent MAT1 interpolation.
TEMPD	Direct	SID, T	Default temperature associated with a temperature set id. This value is applied to nodes omitted from the referenced TEMP set.

Table 15: Analysis-control, parameter, and matrix-input cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
EIGRL	Direct	SID, V1, V2, ND	Real-eigenvalue extraction request used by SOL 103 and SOL 105; the current solver uses the id, lower/upper range, and requested mode count.
PARAM	Limited	NAME, VALUE	Generic parameter storage. All parsed PARAM entries are mapped into model["PARAM_*"]; the active subset currently includes solver controls such as WTMASS and the nonlinear parameters NLLOADSTEPS, NLMAXITER, NLTOL, NLRESTOL, NLRELAX, NLRESMODEL, NLMETHOD, NLLSMAX, NLLSREDUCE, NLCUTMAX, NLCUTREDUCE, and NLSTEPGROW.
DMIG	Limited	NAME, IFO, TIN, TOUT, GJ, CJ, GI, CI, AI	Direct matrix input. The current implementation parses square/symmetric real matrix entries at grid and component level and injects them into the assembled global matrix.

Table 16: Optimization cards currently recognized by the repository.

Card	Status	Fields read	Field meaning in the current implementation
DESVAR	Direct	ID, LABEL, XINIT, XLB, XUB, DELX, DDVAL	Design-variable definition with initial value, bounds, move limit, and optional discrete set id.
DRESP1	Limited	ID, LABEL, RTYPE, PTYPE, REGION, ATTA, ATTB, ATTI...	Response definition. The current parser stores the general descriptor fields, while the routed SOL 200-lite path only supports the narrow subset needed for the present dispatchers, namely mass/weight, compliance, displacement, and buckling-style responses on the documented routes.
DVPREL1	Direct on routed subset	ID, TYPE, PID, PNAME/FID, LB, UB, C0, (DESVAR, COEF)...	Property-to-design-variable relation. The current routed subset relies on these cards for supported shell-thickness and bar-area sizing definitions, including tied groups in which one supported DESVAR drives several routed relations that are preserved as one grouped variable through the translated solve.
DVMREL1	Limited	ID, TYPE, MID, PNAME/FID, LB, UB, C0, (DESVAR, COEF)...	Material-to-design-variable relation. Parsed and stored in the model, with narrower execution support than DVPREL1 in the current SOL 200-lite path.
DCONSTR	Direct on routed subset	ID, RID, lower allowable, upper allowable, LOWFQ, HIGHFQ	Design constraint tied to a DRESP1 response. The routed subset currently uses upper-bound mass constraints on the translated compliance/buckling path, one or more upper-bound compliance or displacement constraints on the one-group exact mass-minimization path, and one or more upper-bound compliance or displacement constraints on the current multi-group routed mass-minimization path.
DOPTPRM	Limited	PARAM, VALUE pairs	Generic optimizer-parameter card. The parser stores name/value pairs and the routed subset currently consumes the parameters it knows how to interpret, including DESMAX, CONV1, and now DELX as the default grouped move limit when routed DESVAR entries leave DELX blank.

E JSON Solver Input Schema and Parser Workflow

JFEM exposes two closely related JSON interfaces, and this distinction is important for anyone who wants to drive the solver without going through the text-deck reader every time. The first interface is a structured raw JSON representation, produced by deck-to-JSON parser and consumed by `json_to_model`. The second interface is the solver-ready model dictionary produced directly by `bdf_to_model`, by `bdf_to_model_json`, or by `json_to_model` after its derivation step. In other words, the raw JSON is already normalized at the card-field level, but it intentionally stops short of computing derived analysis data such as completed isotropic material constants, laminate constitutive matrices, recovered bar-section properties from `PBARL`, merged `SPC`/`SPC1` lists, synthetic `RBE2` entries created from `RBAR`, and resolved coordinate bases.

E.1 End-to-end workflow

Figure 6 shows how the two JSON layers fit into the repository-level dataflow. The direct BDF path resolves INCLUDEs, converts MYSTRAN variants into the normal parser dialect, splits case control from bulk data, tokenizes cards, and then immediately constructs the solver-ready model. The alternate interchange path stops after writing a structured raw JSON file with three top-level objects: `case_control`, `parameters`, and `bulk_data`. That raw file can later be turned back into a solver-ready model through `json_to_model`, which internally calls `build_model_from_json`.

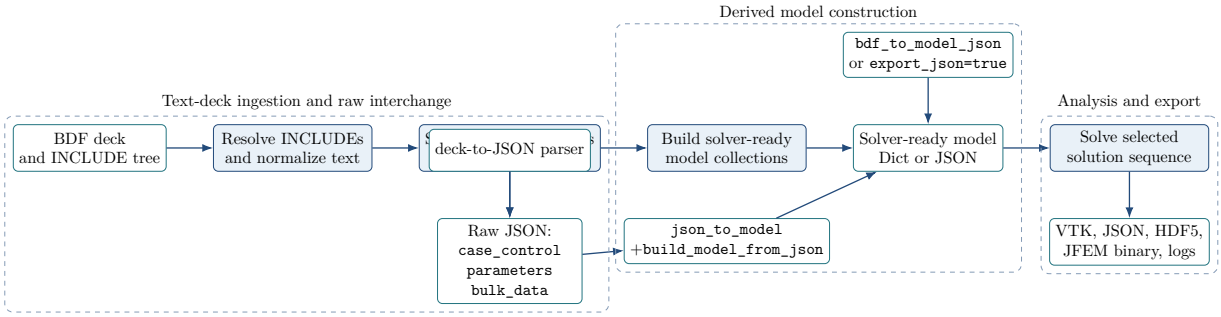


Figure 6: Detailed workflow from a Reference-style BDF deck to raw JSON, the derived solver-ready model, and the final solution/export stages. The raw JSON bridge is intentionally placed before the code computes derived material, laminate, coordinate, and constraint data.

For practical use there are therefore two public model-reading entry points:

```
# Read a raw JSON file emitted by a preprocessor
# conforming to the schema below:

model = json_to_model("model.raw.json"; export_json=true)
results = solve_model(model)

# or, when the full parser path is acceptable:
model = bdf_to_model("model.bdf"; export_json=true)
```

The first route is most useful when a separate pre-processor wants to emit a stable, card-normalized interchange file. The last route is the broadest path in the current repository, because it reflects the full checked-in BDF parser and writes the already derived model that the solver consumes directly.

E.2 Structured raw JSON contract

The raw JSON contract is intentionally small at the top level. It is not a dump of original card text. Instead, it is an object graph of already normalized card fields, organized so that `build_model_from_json` can reconstruct the same major model collections that `build_model` creates from a BDF deck.

Table 17: Top-level keys in the raw JSON file consumed by `json_to_model`.

Key	Required	Typical type	Meaning
`case_control`	Yes	JSON object	Executive and case-control data such as `SOL`, global `LOAD`/`SPC`/`MPC`, modal requests, and the `SUBCASES` object. Because JSON object keys are strings, subcase ids are serialized as strings and converted back to integers inside `build_model_from_json`.
`parameters`	No	JSON object of scalars	Parsed `PARAM` values keyed by uppercase parameter name, for example `WTMASS` or nonlinear controls such as `NLLOADSTEPS`. During derivation these become top-level `PARAM_*` entries in the solver-ready model.
`bulk_data`	No	JSON object of card groups	One child per supported bulk-data family, using card names such as `GRID`, `MAT1`, `PSHELL`, `CQUAD4`, `FORCE`, or `SPC1`. Each child stores card fields in a normalized per-card schema rather than original BDF columns. For the current checked-in exporter, `GRDSET` defaults are already folded into the emitted `GRID` records instead of being stored as a separate child block.

Listing 11 shows a representative minimal raw JSON file. The example is schematic rather than exhaustive, but it matches the actual conventions used by the parser and by `build_model_from_json`: ids are stored as JSON object keys for most entity families, load and constraint cards that may repeat under the same set id are stored as arrays, and coordinate or laminate cards retain only the raw data needed to reconstruct derived quantities later.

Listing 11: Representative minimal raw JSON file for the `json_to_model` path.

```
{
  "case_control": {
    "SOL": 101,
    "LOAD": 1,
    "SPC": 1,
    "SUBCASES": {
      "1": {
        "LOAD": 1,
        "SPC": 1
      }
    }
  },
  "parameters": {
    "WTMASS": 1.0
```

```
  },
  "bulk_data": {
    "GRID": {
      "1": {"ID": 1, "CP": 0, "CD": 0, "PS": "123456", "SEID": 0, "X": [0.0, 0.0, 0.0]},
      "2": {"ID": 2, "CP": 0, "CD": 0, "PS": "", "SEID": 0, "X": [1.0, 0.0, 0.0]}
    },
    "MAT1": {
      "10": {"MID": 10, "E": 7.1e10, "NU": 0.33, "RHO": 2700.0}
    },
    "PSHELL": {
      "20": {"PID": 20, "MID": 10, "T": 0.002}
    },
    "CQUAD4": {
      "100": {"ID": 100, "PID": 20, "NODES": [1, 2, 3, 4], "THETA": 0.0}
    },
    "FORCE": [
      {"TYPE": "FORCE", "SID": 1, "GID": 2, "CID": 0, "Mag": 100.0, "Dir": [1.0, 0.0, 0.0]}
    ],
    "SPC1": [
      {"SID": 1, "C": "123456", "NODES": [1]}
    ]
  }
}
```

The bulk-data child groups follow a few repeated storage patterns:

Table 18: Representative storage conventions inside `bulk_data`.

Pattern	**Examples**	**Stored form**	**Reason**
Entity dictionaries keyed by id	`GRID`, `MAT1`, `MAT8`, `PSHELL`, `PCOMP`, `CQUAD4`, `CBAR`, `EIGRL`	JSON object keyed by stringified ids	These families are naturally looked up by a unique id during derivation and assembly.
Repeated-entry arrays	`FORCE`, `MOMENT`, `PLOAD4`, `PLOAD2`, `PLOAD1`, `GRAV`, `LOAD`, `SPC1`, `SPC`, `MPC`	JSON array of objects	Multiple rows can share one set id, so an append-only sequence mirrors the original card semantics more naturally than a single keyed object.
Set-combination maps	`SPCADD`, `MPCADD`	JSON object keyed by set id	Each card stores one set id together with a list of referenced subordinate sets.
Special nested objects	`CORD2R`/`CORD2C`/`CORD2S`, `RBE3`	JSON object with card-specific nested arrays or point data	These families carry structured subdata such as defining points `A`/`B`/`C`, or multiple weight groups.

At the field level, the checked-in raw-json producer retains only the card data needed to rebuild the analysis model. For example, `MAT1` stores only the raw elastic constants that were present in the deck, without completing missing `E`/`G`/`NU`; `PSHELL` stores thickness and material links, but not a precomputed constitutive matrix; `PCOMP` stores only the declared plies and laminate flag, without expanding `SYM`; `CORD2*` cards store the defining points rather than the final orthonormal basis vectors; and `GRID` entries can already carry inherited `PS`/`SEID` data because the current exporter resolves `GRDSET` defaults before writing the raw JSON bridge.

E.3 Derived solver-ready model

Figure 7 shows the boundary between the raw JSON contract and the solver-ready model that the finite element solver actually consumes. The central operation is `build_model_from_json`. It completes materials, expands or normalizes several property and constraint families, builds the canonical element/property/material collections, resolves coordinate systems, transforms geometry, and injects `PARAM_*` controls and optional optimization metadata.

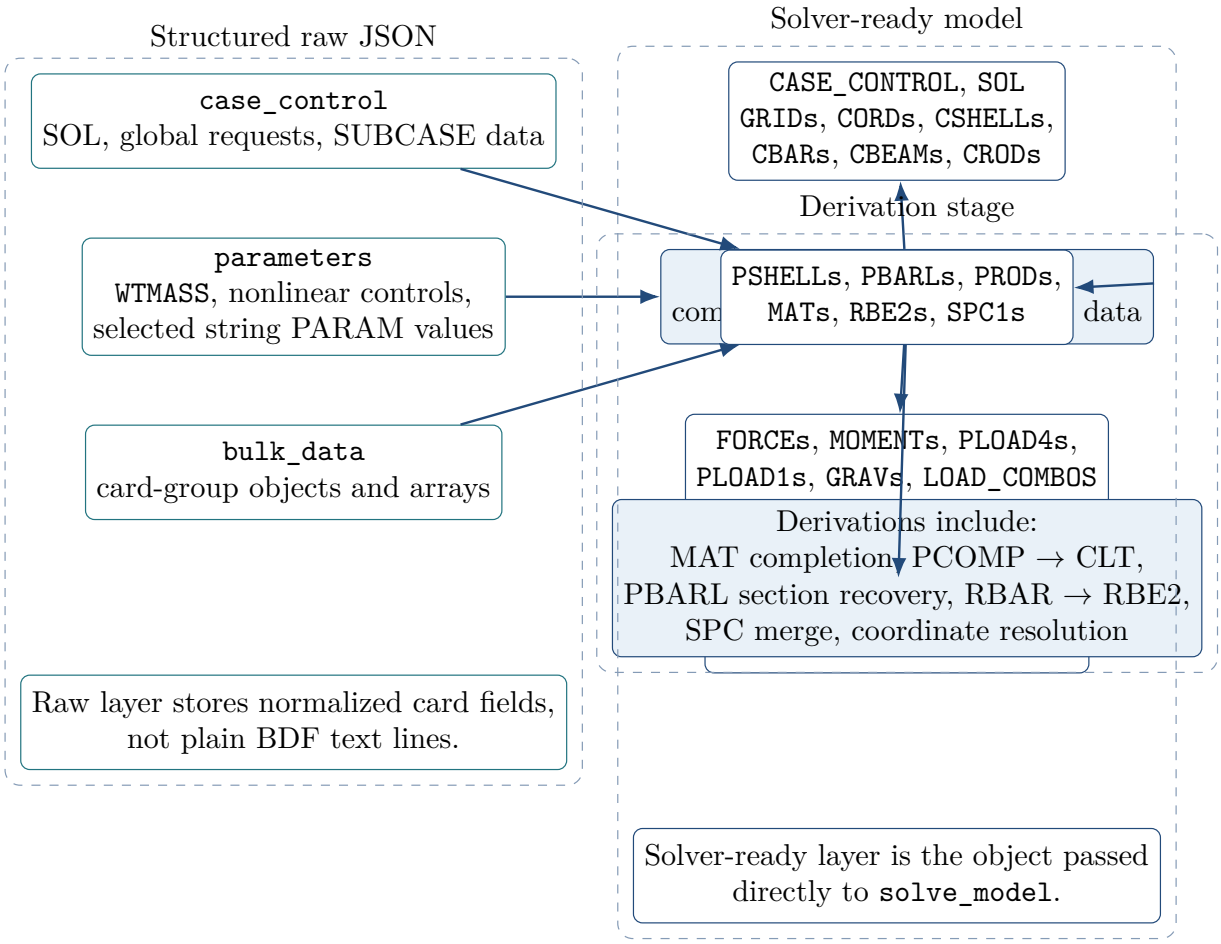


Figure 7: Layered view of the raw JSON schema and the derived solver-ready model. The raw JSON stores normalized card fields; the solver-ready model stores analysis collections and derived data needed by assembly and solution.

The main derivations performed during the raw-to-model reconstruction are summarized in Table 19.

A representative solver-ready model therefore has top-level groups such as:

```
{
  "CASE_CONTROL": {...},
  "SOL": 101,
  "GRIDs": {...},
  "CORDs": {...},
  "CSHELLs": {...},
  "CBARs": {...},
```

Table 19: Key derivations performed by `build_model_from_json`.

Derivation	Effect in the solver-ready model
`MAT1` completion	Missing isotropic constants are recovered from the available `E/G/NU` pair when possible, and the final materials are collected in `MATs`.
`MAT8`/`MAT2` compatibility views	Orthotropic and anisotropic materials receive the compatibility fields required by downstream solver paths that expect scalar `E/G/NU`-style access.
`PSHELL` defaulting	Blank bending or shear fields are filled with repository defaults so that shell constitutive data are always available in a consistent form.
`PCOMP` laminate reduction	Ply lists are expanded when `LAM=SYM`, classical laminate theory matrices are assembled, an equivalent synthetic material id is created, and the result is stored in the shell-property collection used by the solver.
`PBARL`/`PBAR`/`PBEAM` normalization	Section dimensions or explicit section data are converted into the common internal bar/beam property object written under `PBARLs`.
`RBAR` normalization	Each `RBAR` is turned into an `RBE2`-style entry so the solver sees a single rigid-interpolation representation.
`SPC` + `SPC1` merge	Explicit and set-style SPC definitions are combined into one `SPC1s` list for the downstream boundary-condition path.
Coordinate resolution	Raw `A/B/C` coordinate-definition points become orthonormal `Origin/U/V/W` frames, after which geometry is transformed into the solver's working basis.
`PARAM` promotion and optimization packaging	Parsed `PARAM` values become top-level `PARAM_*` entries, and when the relevant cards and case-control requests exist, a packaged `OPTIMIZATION` object is added.

```
  "PSHELLs": {...},
  "PBARLs": {...},
  "MATs": {...},
  "FORCEs": [...],
  "PLOAD4s": [...],
  "SPC1s": [...],
  "EIGRLs": {...},
  "PARAM_WTMASS": 1.0
}
```

This is the object that `solve_model` expects. It is also the JSON written by `bdf_to_model(...; export_json=true)` or returned by `bdf_to_model_json`. For external applications this solver-ready JSON is currently the safer interchange form, because it reflects the broad direct BDF import path used by the main solver entry point.

E.4 What the checked-in raw-json bridge covers today

The raw-json path is useful, but the code base makes clear that it is not yet the broadest import route in the repository. The checked-in producer script deck-to-JSON parser writes the families it knows how to emit in normalized raw form, and `build_model_from_json` reconstructs the analysis collections it explicitly supports. That subset already covers the central shell, bar, beam, rod, spring, mass, rigid, load, SPC, MPC, and modal-request families needed by the primary static, modal, buckling, and current nonlinear shell workflows. However, the direct `bdf_to_model` path remains broader in the current source tree, because it is wired directly into the full parser and builder stack used by day-to-day solving.

For that reason the recommended rule is simple:

- Use raw JSON when a pre-processor wants a compact, normalized, card-level interchange file and the target workflow stays within the families reconstructed by `build_model_from_json`.
- Use the solver-ready JSON written by `bdf_to_model` when the goal is maximum fidelity to the current repository support matrix.

E.5 Parser stages that create the JSON

The parser workflow that produces either JSON layer can be summarized in six concrete stages:

1. **Line ingestion and INCLUDE resolution.** The `resolve_includes` helper recursively replaces `INCLUDE` cards with the contents of the referenced files, preserving nested deck organization while giving later stages one unified line stream.
2. **Dialect normalization.** `convert_mystran_to_reference` rewrites accepted MYSTRAN-specific variants into the internal BDF-based form expected by the remaining parser code.
3. **Executive and case-control parsing.** `read_bulk_and_case` reads `SOL`, global requests, explicit `SUBCASE` blocks, and the `BEGIN BULK`/`ENDDATA` partition. If no explicit subcase exists but global loading or SPC data are present, the code synthesizes `SUBCASE 1`.
4. **Card tokenization.** `process_cards` converts fixed-field, free-field, tab-delimited, and large-field entries into grouped card arrays, while also following continuation lines. This stage still works in terms of tokenized card fields rather than derived engineering objects.
5. **Card-level normalization.** Card-specific extractors convert tokenized fields into structured dictionaries or arrays with named entries such as `ID`, `PID`, `MID`, `NODES`, `THETA`, `Dir`, or `PLIES`. The checked-in script deck-to-JSON parser stops here and writes the resulting object graph as raw JSON.
6. **Analysis-model derivation.** `build_model` or `build_model_from_json` turns those structured card groups into the solver-ready collections used by assembly and solution, then resolves nested coordinate systems and transforms geometry before `solve_model` is called.

The key conceptual point is that JFEM does not consume plain BDF syntax at the solver boundary. By the time the finite element routines are called, the input has already been converted into a structured model object with explicit geometry, property, material, load, constraint, and case-control collections. The appendices above document the card-level language accepted by the parser; the present appendix documents the JSON layer that sits between that parser and the actual solver.

F Validation Process, Parity Reports, and Statistics

JFEM uses a layered validation strategy rather than relying on one monolithic benchmark number. The public repository carries a curated validation suite under `validation/`, while private reference decks, commercial-reference comparisons, run outputs, and developer-only probes live under the validation archive. This appendix documents those layers, the grading rules used by the broad private corpus campaigns, the current regenerated broad-corpus summary, and the archived detailed snapshot tables retained in the private workspace.

F.1 Validation layers and recorded artifacts

The broad private corpus workflow uses decks and references under the model archive, current developer tools under the validation harness, and timestamped campaign outputs under the comparison archive. The workflow discovers same-stem `.bdf`/`.f06` pairs in the validation corpus, builds a solver context from each deck, dispatches the run by solution family, parses the matching F06 reference tables, computes analysis-specific parity metrics, assigns a coarse grade, and records one machine-readable result per deck. The two primary artifacts are:

- `report.ndjson`: one JSON record per processed deck, including the deck name, detected solution type, status, comparison payload, grade, wall time, and any captured exception.
- `summary.json`: an aggregated roll-up of status counts, solution-family counts, and selected named model lists.

Current campaign summaries are kept with the run folder, normally as `summary.json`, `summary.csv`, or a short run report. Those regenerated summaries carry richer machine-readable roll-ups than the older archived snapshot alone, including both raw and effective status counts, canonical-solution counts, grade distributions by analysis kind, metric percentiles, and top unsupported or solver-error classes.

Figure 8 summarizes the broad-corpus path and its relationship to the focused regression layer.

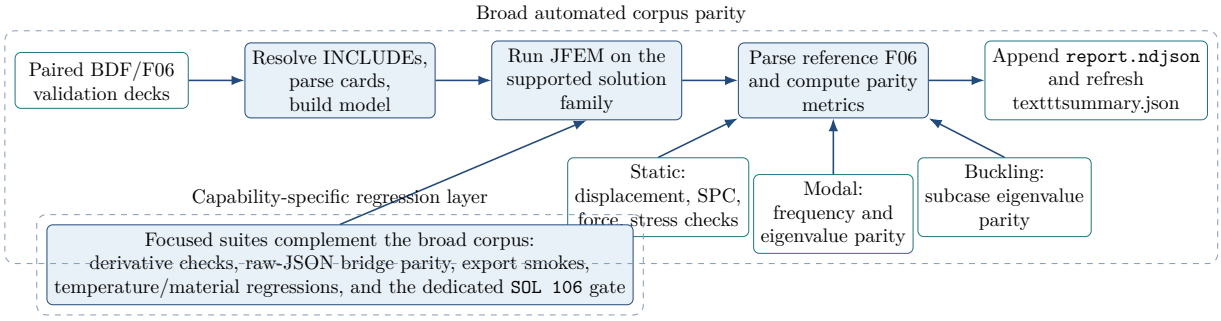


Figure 8: Validation workflow used by the repository. Broad automated corpus parity is recorded deck-by-deck in NDJSON and summarized in JSON, while focused regression suites guard specific capabilities that a broad corpus snapshot may miss or under-sample.

The focused layer complements the broad corpus in several important ways:

- derivative and adjoint checks compare analytical, exact laminate/shell, AD-backed, or finite-difference gradients on controlled decks,

- raw-JSON bridge parity tests compare the direct BDF path with the `BDF → raw JSON → model` route on both synthetic and real decks,
- export smoke tests ensure JSON and HDF5 outputs exist and carry the expected payload structure for the supported solution families,
- targeted thermal/material tests exercise temperature-dependent `MAT1/MATT1/TABLEM1` behavior on line, shell, solid, and modal workflows,
- dedicated private route checks provide a focused readiness gate for the currently supported nonlinear `SOL 106` subset.

F.2 Parity metrics and grading rules

The comparison harness does not use one universal scalar across every solution family. Instead, each supported analysis kind has its own parity payload and one main grading metric. Table 20 summarizes the current rules.

Table 20: Parity metrics used by the broad validation harness.

Analysis kind	**Quantities compared**	**Primary grade driver**	**Current implementation detail**
Static (`SOL 101`)	Grid displacements, SPC forces, selected element-force components, and selected shell-stress summaries.	Maximum relative displacement error over significant compared DOFs.	A displacement component is considered significant only if the reference magnitude exceeds $\max(10^{-10}, 10^{-8} \max \lvert u_{\mathrm{ref}} \rvert)$; the deck grade is driven by the worst retained displacement component even though the report also stores SPC-, force-, and stress-parity payloads.
Modal (`SOL 103`/legacy `SOL 63`)	Real eigenvalues and natural frequencies, now tracked per modal subcase.	Maximum matched frequency relative error.	The current harness matches F06 modal data per subcase and also records eigenvalue errors. The archived broad snapshot tabulated below predates this upgrade, so its modal corpus counts intentionally understate the current harness.
Buckling (`SOL 105`)	Real buckling eigenvalues grouped by subcase.	Maximum matched eigenvalue relative error.	Matching is sign-aware: for each paired mode the harness uses $\min(e_-, e_+)/\lvert\lambda_{\mathrm{ref}}\rvert$ where $e_\pm = \lvert\lambda_{\mathrm{test}} \pm \lambda_{\mathrm{ref}}\rvert$, so shear-style $\pm\lambda$ pairs are not penalized for sign convention alone.

The qualitative grade thresholds are common across the supported parity modes and are listed in Table 21.

Table 21: Grade thresholds used by the broad validation harness.

Grade	Threshold	Meaning
EXCELLENT	error $< 10^{-3}$	Sub-percent parity, typically at or near the level expected for the closest matched benchmark cases in the current harness.
`GOOD`	error $< 10^{-2}$	Strong practical agreement for the present comparison metric.
`FAIR`	error $< 5 \times 10^{-2}$	Moderate agreement; often acceptable for broad-corpus trend tracking but not a claim of tight solver parity.
`POOR`	error $\geq 5 \times 10^{-2}$	Large discrepancy on the selected grade-driving metric.
`NO_DATA`	no comparable reference values	The deck ran, but the selected grade-driving metric had no finite matched values to compare.

F.3 Completed broad-corpus snapshot

The latest completed broad-corpus report is a private validation artifact under the comparison archive. Table 22 gives the headline status counts from that private campaign. The percentages are with respect to all 3997 paired decks in the completed run.

The deeper family-, grade-, unsupported-, and error-breakdown tables that follow still use the archived detailed private snapshot in the detailed comparison archive, because that archive remains the most stable source for those per-class examples. Table 23 therefore keeps its archived family breakout as a historical detailed reference alongside the newer headline counts above.

The archived grade distributions for the completed `OK` decks are shown in Table 24. These counts are intentionally conservative because each deck is graded by its single worst primary comparison metric.

Tables 25 and 26 summarize where the archived broad-corpus limits are concentrated.

F.4 Representative parity reports

Table 27 gives three concrete parity examples that are easy to inspect in the repository. The first two come from the completed archived broad-corpus snapshot. The third is a post-upgrade modal spot-check from the newer subcase-aware harness, included because the archived snapshot predates the present `SOL 103` comparison path.

Table 22: Headline statistics from the current regenerated broad-corpus summary.

Category	Count	Percent	Interpretation
Total paired decks	3997	100.0%	Aggregate BDF/F06 corpus discovered by the runner.
`OK`	2048	51.2%	Decks completed and produced a comparison payload in the current run.
`SOLVER_ERROR`	973	24.3%	Raw solver-error count before deck fragments are normalized out.
`UNSUPPORTED_SOL`	976	24.4%	Decks requested solution families outside the scope of the current broad-corpus run.
`DECK_FRAGMENT` (effective only)	267	6.7%	Standalone INCLUDE fragments or incomplete paired decks separated from genuine solver failures by the regenerated summary.
Completion within supported workflows (raw)	2048 / 3021	67.8%	Successful runs divided by `OK + SOLVER_ERROR` before fragment normalization.
Completion within supported workflows (effective)	2048 / 2754	74.4%	Successful runs divided by `OK + effective solver errors`; this is the more faithful current breadth number.

Table 23: Solution-family status in the latest completed archived broad-corpus snapshot.

Solution	Total decks	OK	Other status	Interpretation
`SOL 101`	940	682	258 solver errors	Broadest linear-static slice in the archived run.
`SOL 103`	156	0	156 unsupported	Archived snapshot predates the current subcase-aware modal comparison path. These decks are no longer a reliable measure of present modal capability.
`SOL 105`	1915	1908	7 solver errors	Most mature broad-corpus family in the archived run; strongly dominated by shell buckling decks.

Table 24: Grade distribution of completed decks in the archived broad-corpus snapshot.

Family	Excellent	Good	Fair	Poor	No data	Interpretation
Static (`SOL 101`, 682 OK)	113	5	8	447	109	The static corpus is broad and harshly graded by worst retained displacement component; many decks still run but expose large parity gaps or sparse comparable reference payloads.
Buckling (`SOL 105`, 1908 OK)	5	122	1492	251	38	Buckling coverage is numerically broad, with most decks clustering in the `FAIR` band and a much smaller tail in `POOR`.

Table 25: Most frequent unsupported solution families in the archived broad-corpus snapshot.

Solution code	Count	Interpretation
103	156	Modal decks left unsupported in the archived snapshot before the recent harness upgrade.
166	119	Out-of-scope aeroelastic family for the archived run.
199	81	Out-of-scope advanced or optimization-style family in the archived corpus.
153	73	Out-of-scope frequency-response or dynamic family in the archived corpus.
106	70	Nonlinear decks not represented in the broad archived corpus summary, even though targeted `SOL 106` gates now exist separately.
161	62	Additional out-of-scope dynamic family.

Table 26: Most frequent solver-error classes in the archived broad-corpus snapshot.

Error message prefix	Count	Typical interpretation
`No nodes found in model -- is this a standalone BDF or an INCLUDE fragment?`	252	The paired deck is effectively an INCLUDE fragment or otherwise not directly solvable as a standalone model in the current runner.
`SingularException(0)`	11	Nominally supported route reached a singular reduced system.
`StackOverflowError:`	10	Recursive or deeply nested failure on a small unsupported corner of the archived run.
`TaskFailedException`	1	Solver task failed without a more specific top-level classification.
`BoundsError: attempt to access 2x3 Matrix{Float64} at index [3, 1:3]`	1	Isolated parser or post-processing bug in the archived run.

Table 27: Representative parity cases from the repository validation workflow.

Deck	Route	Recorded parity	Grade	Meaning
line element	Static broad-corpus snapshot	Maximum displacement relative error 5.89×10^{-16}	EXCELLENT	Small line-element benchmark with essentially exact static parity at the grade-driving displacement metric.
laminate shell	Buckling broad-corpus snapshot	Maximum eigenvalue relative error 2.30×10^{-4} over five matched subcases	EXCELLENT	Composite-laminate shell buckling deck showing close agreement on the first matched buckling factor per subcase.
solid modes	Updated modal harness smoke	Maximum matched frequency relative error 1.23×10^{-2}; matched subcases 1 and 5	`FAIR`	Real solid-modal deck used to confirm that the newer subcase-aware modal path produces measurable F06 parity rather than leaving `SOL 103` decks unsupported.

F.5 Complementary targeted validation suites

The broad corpus is only one layer. Table 28 summarizes the capability-specific suites that currently fill the main gaps left by a deck/F06 comparison alone.

Taken together, these layers give a more useful picture of JFEM than any one scalar success count. The archived broad-corpus snapshot is still the best completed quantitative breadth measure in the repository, but the focused suites are what currently carry the highest-confidence evidence for nonlinear behavior, parser/export fidelity, temperature-dependent materials, raw-JSON transport, and the newer modal features that postdate the archived broad-corpus report.

Table 28: Targeted validation suites that complement the broad deck/F06 corpus.

Suite or script	**Scope**	**Current recorded role**
Public validation suite	Publishable correctness examples	The checked-in `validation/public_suite.yaml` and `validation/run_public_suite.jl` exercise public classical, MacNeal-Harder, MYSTRAN cross-reference, and CRM-derived decks without exposing private industrial cases.
Private broad validation campaigns	Commercial-reference parity and regression history	Private decks and references are held under the model archive; campaign results are archived under the comparison archive.
Focused private validation tools	Capability-specific probes	Current developer tools under the validation harness cover F06 displacement comparison, singularity summaries, shell geometry audits, PCOMP formulation audits, SOL 101 feature guards, and SOL 105 static-preload parity.
Private code route checks	Backend and nonlinear readiness	Private code tools under the development harness include TACS/SOL 106 route checks used to guard backend-integration assumptions without shipping external payloads in the public repository.
Public runner and worker checks	User-facing execution paths	The public entry points under `__Start here!__/julia_tools/` define the maintained single-deck runner, batch-manifest runner, text-list batch runner, and persistent JSONL worker used by wrappers and automation.